Custody Theory: The Mathematics of One-Sided Certification over Adversarially Held Records

The mathematics the identity, scrutiny, and standing machines are instances of.

Roque Briceño
Version 0.8.1. 4 September 2026.

Drafted with AI assistance from the kernel and the three layer papers, and adopted by the author for this release.

Companion reports: *Fail-Closed Class Dispatch* (identity), *Refutation-Gated Admission* (scrutiny and standing), and *Admissible* (the composition). Those papers prove theorems about three machines. This paper asks what the three machines are instances of, states the theory, and reads the theory back into the kernel as a list of things the kernel could compute and does not yet. Where this paper and the kernel disagree, the kernel is the source of truth and this paper is wrong.

Abstract

Admissible certifies work-products of non-replayable generators by a single query over an append-only record, admissible(id), and proves two things about it: that a positive answer is re-derivable from the record (soundness of the record) and that every attempt to falsify it either writes nothing or writes a named fault first (loudness of deviation). Its three layers — identity, scrutiny, standing — are each a guarded fold over a journal, composed by disjoint write sets; its every carried quantity is a primary (a count, a set, a hash), refused promotion to a rate; its composition of measured power is by union and max, never by product; its escapes are counterfactual re-runs of instruments the seal already trusted; and its one acknowledged residue is that deleting the right events *raises* standing. Each of these is, in the layer papers, a design decision defended by adversarial review and enforced by a mutation-tested guard.

We show they are theorems of one structure. A **custody** is an append-only record held by parties who may lie, together with a semantics under which the record certifies exactly what holds in every world consistent with it under a declared set of trust assumptions — the **custodial reading** — and a second, disjoint kind of certification, the **demonstration**, which is a self-verifying proof object the record contains. Certification splits by polarity: necessities are monotone in the record and re-derivable by replay; demonstrations are monotone in the record and *not* re-derivable, because the absence of a proof is not a proof of anything. From that split follow, as theorems rather than decisions: why a standing predicate is a conjunction of necessities and negated demonstrations and therefore non-monotone; why deletion, and the insertion of a degrader, are the tampering directions that raise standing, and which events they must target; what any anchoring scheme must pin and the minimal certificate that pins it; why the sound assumption-free composition of measured power is exactly the Fréchet family (union on a shared defect model, max across models, Bonferroni for conjunctions) and why power_min, read as a joint figure, is an upper bound wearing a lower bound's name; why temporal precedence witnessed by a hash preimage survives every coherent rewrite of the journal and precedence witnessed by a journal position does not; why invariants pull back along the projection from a stacked machine to the one beneath it; and when a finite set of deleted guards certifies a universally quantified invariant. The theory yields twenty-eight concrete kernel capabilities, many of them pure queries over journal state the kernel already keeps, each anchored to a code site, and fourteen findings about the kernel as it is — each predicted by a theorem, executed against the unmodified code, and signed by its direction. One of them — a single-party report that un-impeaches a line, through either of two channels, against one row of the standing layer's own transition table — is the theory's clearest

earning. It also inherits the layer papers' limits exactly: nothing here converts procedure into truth, a primary into a rate, or a survived attack into correctness.

0. How to read this paper

Sections 1–2 fix the objects: records, machines, the replay language. Section 3 is the centre of the paper — custodial semantics, polarity, and the Asymmetry theorem; it forward-references D26 (§3.6) and the move set of §5, and is best read with them. Sections 4–7 are the four facets that the Asymmetry theorem organizes: the algebra of carried doubt (§4), the geometry of the record under rewriting (§5), stacked custody and adequacy (§6), and the custody game (§7). Section 8 reads the theory back into the kernel as capabilities, each tiered by cost; §8a is what executing the theory against the kernel found; REVIEWS.md is the record of the nine review rounds that corrected this paper: three adversarial referee rounds, the pull request's automated review, two seven-referee re-reviews of the pull request head, and three internal six-reviewer adversarial re-reviews. Section 9 is the novelty ledger: which parts are known mathematics wearing new names, which are genuinely new, and which are conjectures. Section 10 is the limits. Every definition is numbered D, every axiom X, every theorem T, every kernel correspondence K. Citations into the kernel are file:symbol; the kernel's own theorem labels (I1–I17, R1–R13, C1–C7, faults F/V/E, assumptions A0–A13, B0–B14) are used unchanged.

A convention inherited from the layer papers: a plain reading may never weaken a guarantee, and every theorem is followed by what it does not say.

1. The problem, restated as mathematics

The layer papers state the problem as engineering: a generator's identity no longer determines its output, so certification must attach to a procedural record rather than to an author. Stated as mathematics, the problem has four parts, and each existing layer answers exactly one of them by construction.

The record is the only object. Nothing is known about the artifact except through events that untrusted parties appended to a journal, under guards the kernel enforced. A consumer who asks admissible(id) is asking a question about a word in a free monoid.

Certification must be one-sided. The consumer must never receive a false positive from any sequence of adversarial writes; the adversary may, at worst, cause silence (a refusal, a closed line, an unreachable gate). This is the fail-closed discipline, and it is a statement about the *variance* of the answer under adversarial action, not about its value.

Every carried quantity must compose without an assumption it does not carry. Measured power is exact on a named finite defect model and nothing else; the kernel refuses to multiply powers because multiplication assumes independence nobody certified. What the kernel needs is not a probability calculus but the calculus of what is sound under *every* coupling consistent with the carried primaries.

The record itself is held by an adversary. Replay recognizes whether a journal is one the machine would have produced; it cannot recognize which such journal actually was produced. The layer papers enumerate the consequences (coherent root rewrite, tail truncation, recompute-free deletion, endpoint stamp forgery) and bolt on an authenticated-publication receipt. What is missing is the characterization: the space of coherent alternatives, its structure, and what any anchor can and cannot pin.

The four parts are not independent. The same polarity that makes certification one-sided (§3) decides which events an anchor must pin (§5), which compositions of power are sound (§4), and which guards survive rewriting (§5, §7). That single polarity is the content of this paper.

1.1 On the word stochastic

The layer papers call the generator *stochastic* ("the generator is untrusted and stochastic; the refuter is deterministic and replayable", ../RGA/DRAFT.md §1; "delegation to stochastic generators", ../admissible/DRAFT.md §1.1) and call a refuter that diverges on replay *nondeterministic* ("caught nondeterministic", ../admissible/DRAFT.md §1.2). The first word is true of the mechanism — a language model samples from a distribution — and it is not the property any theorem consumes. Three observations fix the right word.

First, no theorem in I, R or C uses a distribution over the generator's outputs. The only assumption that mentions one is B4 (k samples are k draws from the bind's output distribution), and the only use B4 receives is that a whole-response cache replay is *one* draw presented k times — a statement about distinctness of executions, not about a measure. Every quantity the kernel carries is a primary precisely because the kernel refuses to put a measure on the generator (B6, C5, §9 of the composed paper). The semantics of §3 is accordingly *possibilistic*: [[r]]_B is a set of worlds, not a distribution over them, and []_B is a universal quantifier, not an expectation. A word that imports a probability space the theory declines to use over-describes the object.

Second, the word under-describes the adversary. A stochastic generator is a random one; the threat model is a *strategic* one — "any sequence of public transitions by untrusted parties … tailoring outputs to known tests, witness multiplication, … selection across retries" (../admissible/DRAFT.md §2). The guards are a winning strategy against every behaviour, random or chosen (§7); "stochastic" names the benign case.

Third, the property the theorems actually consume is the one the kernel exploits asymmetrically in B1: "a forged *survived* on a deterministic refuter is a replay divergence when a third party re-runs it, which is more than FCD can say of a forged m_exec". A refuter's claims can be *re-derived*; a generator's cannot. In the vocabulary of §3, a component is **replayable** if its outputs are a function of journaled roots — so every claim about them is a necessity, re-derivable by anyone holding the record — and **non-replayable** otherwise, so that every claim about them is either a demonstration (a trial that was run) or an assumption (a report believed). Generators are non-replayable by nature; refuters are *required* to be replayable and are refused when caught otherwise (R5). That is one dichotomy, not two, and it is the one on which every theorem below turns: the identity layer exists because a non-replayable component's identity does not determine its output; the scrutiny layer exists to convert claims about a non-replayable component into demonstrations by replayable ones; the standing layer exists because those demonstrations can be extended after the fact by anyone holding the replayable instrument.

This paper therefore says **non-replayable** where the layer papers say stochastic, reserves **nondeterministic** for the identity-layer contrast ("a deterministic function's identity determines its output; an agent's does not"), and says **adversarial** when the threat model is the point. It uses *stochastic* only for the sampling mechanism of a language model, which is a fact about the world the kernel does not consult. The recommendation to the layer papers is the same substitution, with one exemption: a "stochastic witness source" whose catch rate is measured on a held-out seeded set (../RGA/DRAFT.md §7, ../RGA/PREMISE.md §3) is exactly a component whose *distribution* is the property consumed, and the word is right there. It is a recommendation about precision, not correctness: nothing they prove depends on the word.

2. Records, machines, replay

2.1 Records

D1 (Event alphabet, record). An *event alphabet* is a finite set Σ of event types together with, for each type, a finite set of payload fields partitioned into *root fields* and *derived fields*. An *event* is a type with a payload. A *record* is a finite word $r \in E$ *over events. Records are ordered by the prefix relation [=; |r| is the length, and the* position* of an event is its index in the word.

In the kernel, Σ is the union of the three journal vocabularies (metrics/SCHEMA.md; protocol/schemas): open, stage, bind, call, decide, accept for identity; rga_declare, rga_measure, rga_bound, rga_open, rga_sample, rga_trial, rga_replay, rga_refuse, rga_seal, rga_close for scrutiny; cal_run, cal_replay, cal_discredit, cal_adjudicate, cal_exclude, cal_install, cal_stamp, cal_close for standing. Root fields are the ones a transition consumes from outside — body_hash at open, the artifact bytes' hash and the nonce at rga_sample, a verdict and witness hash at rga_trial, a finder's nonce at cal_run. Derived fields are the ones a transition computes — on_bind, pi_star, seed, power, composite, track_records. Recorded positions and indices (fcd_position, sealed_at, as_of, run_index) are roots on replay: the journal supplies them and replay range-checks them (T12c), which is why pruning before a mediated seal leaves L_rep (CF11). The distinction is the kernel's own: fcd/core.py:_compare_regenerated checks derived fields against re-derivation and, by its docstring, accepts a coherent rewrite of root fields as "another valid history".

D2 (Machine). A *machine* over Σ is a triple (St, s0, μ) where $\mu : E \to$ *St is a guarded partial left fold: $\mu(\varepsilon)$ = s0, and $\mu(r \cdot e)$ is defined iff a guard g_e(μ(r)) holds, in which case $\mu(r \cdot e)$ = step_e(μ(r)). The* accepted language* $L(\mu) = \mathrm{dom}(\mu)$ is prefix-closed by construction.

D3 (Attempt, refusal, publication). The public interface of a machine is a set of *attempts* Att (the kernel's public methods with their arguments). At a state s, an attempt a has one of three outcomes: it is *refused* — μ appends no event and s is unchanged — or it *publishes* — μ appends a word that contains an event carrying a fault label from a distinguished set $\Sigma_F \subseteq \Sigma$, and no store write (S, S_R, a stamp) precedes that event — or it *succeeds* and appends a word with no fault label. The kernel's published faults are not always fault-first: V1 appends the refuted rga_trial and then rga_close(V1); V4 appends rga_replay, rga_refuse, then a close. What holds of every one is that no store is written before the fault. Σ_F includes rga_refuse and cal_discredit: a post-seal divergent replay appends [rga_replay, rga_refuse] with no V-label on either event, and it publishes in this sense — the refusal names what was found. The kernel's fault tables partition its attempts exactly this way: F5–F10, V6–V15, E4/E6/E7/E9 refuse; F1, F3, F4, V1–V5, E5 publish (../admissible/DRAFT.md §6, Theorem 2).

D4 (Store, query). A *store* is a component of St that is a set written by exactly one attempt and never shrunk (S written by Accept, I8; S_R by Seal, R8). A *query* is a total function $q : L(\mu) \to V$ into a partially ordered *verdict domain* V; a query is *pure* if it is a function of $\mu(r)$ alone (it consults no clock, no environment). Every kernel query — admissible, mediated, tainted, impeached, suspect, demoted, charges, track_record — is pure in this sense, and total on configured classes (demoted refuses an unconfigured class, E9) (rga/core.py, rga/calibration.py, "queries (pure)").

2.2 Replay

D5 (Replay language). Let emit(r) be the word of events the machine appends when re-driving the transitions of r from s0, with derived fields recomputed and the timestamp field excluded. The *replay language* is $L_{rep}(\mu) = \{ r \in L(\mu) : \mathrm{emit}(r) = r \}$. The kernel's three from_events decide the equation emit(r) = r — they re-drive, compare field by field (fcd/core.py:_compare_regenerated; rga/core.py:Admission.from_events; rga/calibration.py:CalibrationAuthority.from_events), and refuse on any difference — so they recognize exactly the *re-derivable* records. That coincides with L_rep only where re-derivability and live producibility agree. The kernel's replay recomputes derived fields but does not re-check every live guard, so from_events is a decidable approximation of L_rep that differs from it at exactly the two seams §8a catalogues: it **accepts** a record outside $L(\mu)$ — hence outside

L_rep — that no live run could produce (a cal_run filed after its checker's refusal, which the live guard forbids but replay does not re-impose: CF14, unsound), and it **rejects** a record in L_rep that a live run did produce (a divergent-replay audit journal replay recomputes differently: CF2, incomplete). Every result below that reads the computed deletion surface (custody.py, whose coherent net is from_events re-derivation) as L_rep therefore holds relative to from_events, the repository's replay operator, and inherits CF2 and CF14 as stated qualifications rather than resting on an idealized exact recognizer.

What D5 does not say. That from_events recognizes the idealized L_rep: it recognizes the re-derivable records, and the identification is exact only away from CF2 and CF14. An idealized replay operator that re-imposed every live guard (closing CF14) and admitted the divergent-replay seam (closing CF2) would recognize L_rep exactly; the kernel's does not, and the repairs are catalogued in §8a (N28), not assumed here.

T1 (Replay is a retraction, not an identification). emit restricted to L_rep is the identity, and $\mu \cdot \text{emit} = \mu$ on $L(\mu)$. Hence replay decides membership in L_rep and recovers the state, and for any two distinct records $r \neq r'$ in L_rep, replay alone cannot decide which was produced.

Proof. The first two claims are the definition of L_rep and the determinism of μ given journaled root fields (nonces are root fields, read from the journal and never redrawn: B9, rga/core.py:Admission.from_events "Nonces and journal positions are read from the journal, never redrawn"). The third is immediate: a decision procedure that consumes only r and accepts both r and r' has no input on which they differ. []

What T1 does not say. Nothing about which records lie in L_rep besides the produced one; that is §5.

2.3 Polarity of events

D6 (Polarity). Fix a query $q : L\_rep \to V$. An event type $\sigma \in \Sigma$ is *positive for q* if for every $r \in L\_rep$ and every event e of type σ with $r \cdot e \in L\_rep$, $q(r) \leq q(r \cdot e)$; *negative for q* if always $q(r) \geq q(r \cdot e)$; *neutral* if both; *mixed* otherwise. A positive type is *strictly* positive if $q(r) < q(r \cdot e)$ for some r, and *enabling* if it is never the event at which q rises but is required by one that is. The polarity of Σ for q is the induced labelling.

The kernel's own predicate has a computable polarity, and the paper's central claims are all about it. For q = admissible (D7 below, rga/calibration.py:CalibrationAuthority.admissible) the labelling is: strictly positive — cal_stamp (the event at which mediated, the last conjunct to become true, flips); enabling — accept and rga_seal, the direct premises of the stamp's guard one step removed, which never lower admissible and without which the stamp cannot be issued (rga_seal is itself strictly positive for the narrower Admission.admissible; D6 reads *enabling* this narrowly — a premise of the rising transition's own guard — not as every event some guard on the path requires, which would label most of the vocabulary e); negative — cal_replay establishing a refuted run (via impeached), cal_adjudicate with decision accept; strictly positive by a second-order route — cal_discredit, which never lowers admissible of any line and raises it for every line impeached only by that checker's escapes (and otherwise admissible), since discredited enters admissible only through _check_valid; mixed — rga_refuse, negative for every seal that pinned the refuter after sealing (tainted) and positive for every line impeached only by that checker's escapes; neutral — everything else at the granularity of admissible (open, stage, bind, call, decide, rga_declare, rga_measure, rga_bound, rga_open, rga_sample, rga_trial, rga_close, rga_replay with agreement, cal_run — filed is not established — cal_exclude, cal_install, cal_close). The second-order types are what D6 is designed to expose: an event negative for q = escapes(cls) is positive for q = ¬impeached(id). T3 handles it; §8a, CF1, is what it costs. The companion's table (custody.py:POLARITY) uses e for enabling and + for strictly positive.

3. Custodial semantics

3.1 Worlds and the custodial reading

The layer papers repeat one sentence in every section that touches a number: *the reading is the reader's, not the kernel's* (B6). This section makes the sentence a definition.

D7 (World, consistency, trust base). A *world* w is a complete history of what actually happened: which model ran each call, whether each harness report was true, whether each hash was collision-free, whether the artifact satisfies each claim, what the generator's context contained. W is the set of worlds. A *trust assumption* is a subset $\alpha \subseteq W$ (the worlds in which it holds); the kernel's assumptions A0–A13, B0–B14 and the calibration layer's are trust assumptions. A *trust base* B is a finite set of trust assumptions; $\cap B$ is the set of worlds satisfying all of them. The *consistency relation* $\models \subseteq W \times E^*$ holds, $w \models r$, iff every event of r is, in order and with the same root fields, a transition the machine performed in world w — r is a subsequence of the record the world produced. This is the epistemic position of a verifier who holds a record that may have been pruned: what it knows is that these transitions happened in this order, not that nothing happened between them.

D8 (Custodial reading). For a record r and trust base B, the *world-set of r under B* is $[[r]]_B = \{ w \in \cap B : w \models r \}$. A *world property* is a subset $P \subseteq W$. The *custodial reading* of P at r under B is the necessity $[]_B P(r) \Leftrightarrow [[r]]_B \subseteq P$, read: *every world whose history contains this record, under these assumptions, satisfies P*. A query of the form r |-> []_B P(r) is a *necessity query*. Two remarks fix its use. **Vacuity.** L_rep contains records no world in $\cap B$ contains — replay decides consistency of the record with itself, not with any world — and at such a record every []_B P is vacuously true; every theorem below carries the premise $[[r]]_B \neq \emptyset$, which deletion preserves (X1) and which only insertion or rewrite can destroy. **The custodial premise.** A []_B P verdict speaks about the actual world only if the actual world is in [[r]]_B, i.e. the presented record is a true sub-record of the produced one. Deletion keeps that premise (X1); insertion and rewrite break it. That sentence is the Asymmetry theorem before it is proved.

D8′ (Root-determined necessity). For an event type σ and root fields ρ, let $P_{\sigma,\rho} = \{ w :$ the machine performed a σ-transition with roots ρ in w $\}$. A *root-determined necessity* is []_B P for P a finite intersection of such sets. Under D7, []_B P(r) holds non-vacuously only if $P \supseteq \cap B \cap \uparrow r$ ($\uparrow r$ the worlds whose history contains r): certifiable properties are supersequence-closed, i.e. presence claims. is_sealed(id) and mediated(id) are root-determined — the presence of an rga_seal and of a cal_stamp with the roots the seal chain consumed — and every clause below that speaks of "the roots a necessity mentions" (T4b, T4c, T11, T17) is stated for root-determined necessities. What the seal's *guards* checked rides on A0, that the machine that appended the event is the kernel; replay decides $r \in$ L_rep, not that.

The custodial reading is what "fail-closed" means once one asks what a verdict is about. The kernel does not say the artifact is correct; it says that in every world consistent with the record — including the worlds where the generator is adversarial, the finder is adversarial, and every party not covered by an assumption in B lied — the procedure was followed. Theorem 1 of the composed paper ("Soundness of the record") is precisely []_B P_proc (r) for P_proc = "for each sample a bind and an observation agree with ϕ of an admitted specialist; every pinned refuter survived on every sample after a post-artifact seed; …" with B = {A0–A13, B0–B14}.

X1 (Consistency is antitone under sub-records). If $r' \subset r$ as a scattered subword and $w \models r$, then $w \models r'$; in particular $w \models r \cdot e$ implies $w \models r$.

Under D7 this is a one-line lemma rather than an axiom — a subsequence of a subsequence is a subsequence — and it keeps its label because everything below uses it and nothing stronger about $\models$.

T2 (Necessity queries are monotone). For every world property P and trust base B, r [= r' in L_rep implies []_B P(r) ≤ []_B P(r'). Equivalently, r |-> [[r]]_B is antitone from (L_rep, [=) to (P(W), ⊆).

Proof. By X1, [[r·e]]_B ⊆ [[r]]_B; if [[r]]_B ⊆ P then [[r·e]]_B ⊆ P. Induct on the extension. []

What T2 does not say. Nothing about queries that are not necessities, and admissible is not one.

3.2 Demonstrations

admissible cannot be a necessity query, because it goes false when an escape is filed: more record, lower verdict, contradicting T2. The second conjunct kind is a different mathematical object.

D9 (Demonstration). (Not the few-shot sense of the word in the language-model literature the generators come from.) A *demonstration shape* is a set X ⊆ E *of finite words each of which is* self-verifying*: membership of a word in X is decidable from the word and the state at its start, by re-deriving fields the kernel computes and requiring the presence of a witness the kernel checked. A* demonstration query *is d_X(r) ⇔ ∃ x ∈ X . x is a scattered subword of r positioned consistently with μ, read:* the record contains a proof of shape X*.

The kernel's demonstrations are exactly its escapes and its refusals. A tier-A escape is the word (cal_run(verdict=refuted, seed=H(ν|h|r|claim)), cal_replay(diverged=false)) over a sealed line whose seal names h and pins r on claim; the kernel checks the bytes hash to h (single-holder, rga/calibration.py:_guard_run_bytes), derives the seed itself (_guard_run_seed), and requires the identical replay (replay_run). A refusal is (rga_replay(diverged=true), rga_refuse) over a declared refuter. Neither carries any trust assumption the seal did not already carry: the escape uses B1, B2, B7, B10 — the same instrument, the same runtime, the same hash, the same seed discipline — and that is the whole content of C1.

T3 (Demonstrations are monotone, and their negations antitone). d_X is monotone in [= on L_rep; ¬d_X is antitone. A *validity-degraded* demonstration — one whose witness is itself attackable by a later event (cal_discredit of the checker, rga_refuse of the checker) — is a demonstration query composed with a negated demonstration query, and is therefore mixed.

Proof. A scattered subword of r is a scattered subword of r·e. The validity clause is rga/calibration.py:_check_valid read literally: established ∧ ¬discredited ∧ ¬refused ∧ (tier B ⇒ adjudicated accept); each negated conjunct is ¬d for a demonstration shape. []

D10 (Standing). A *standing query* is a conjunction of necessity queries and negated (possibly validity-degraded) demonstration queries: stand(r) = ∧_i []_B P_i (r) ∧ ∧_j ¬ d_{X_j}(r).

K1 (admissible is a standing query). admissible(id) = mediated(id) ∧ is_sealed(id) ∧ ¬tainted(id) ∧ ¬impeached(id) (rga/calibration.py:CalibrationAuthority.admissible, composing rga/core.py:Admission.admissible). is_sealed and mediated are root-determined necessity queries (D8′) under B = {A0–A13, B0–B14}: sealed means [](an rga_seal for this line, with these roots, is present), and that its guards held when it was appended rides on A0; mediated means [](a cal_stamp bound to this seal's position is present). Replay decides that the record is in L_rep; it does not, and cannot, certify that nothing else happened between the events. tainted is a demonstration query (a refusal after sealing of a refuter the seal relied on, rga/core.py:Admission.tainted); impeached is a validity-degraded demonstration query (impeached · _check_valid).

3.3 The Asymmetry theorem

The layer papers observe, as a residue, that "deletion is the direction that can *raise* standing, since dropping an escape un-impeaches its line and dropping a refusal un-taints every seal that relied on it." The observation is a theorem about the two kinds of certification, and it is sharper than the observation: it says which events, and only which, an adversary who deletes can profit from, and what an adversary who inserts must supply — a root the anchor hashes, or a degrader the anchor counts.

D11 (Tampering moves). For r ∈ L_rep, a *deletion* produces a scattered subword r' ⊂ r with r' ∈ L_rep; an *insertion* produces r' ⊃ r with r' ∈ L_rep; a *coherent rewrite* replaces root fields of some events and recomputes every derived field so that r' ∈ L_rep with |r'| = |r|. Every element of L_rep reachable from r by finitely many such moves is a *coherent alternative* to r.

T4 (Asymmetry). Let stand = ∧ []_B P_i ∧ ∧ ¬d_{X_j} be a standing query on L_rep, and let r' be a coherent alternative to r.

(a) *Deletion lowers every necessity and can raise standing only through a negated demonstration.* If r' ⊂ r then []_B P_i(r') ≤ []_B P_i(r) for every i, and stand(r') > stand(r) implies that some x ∈ X_j present in r is absent from r'.

(b) *Insertion raises a root-determined necessity only through roots the anchor pins, and raises standing in two other ways the anchor also pins.* If r' ⊃ r and []_B P_i(r') > []_B P_i(r) for a root-determined P_i, then some inserted event carries a root field P_i names, and that root is a positive atom of the query (D26): the kernel's instance is a cal_stamp inserted for an unmediated seal, whose roots (line_id, sealed_at) replay checks and whose insertion — at the endpoint, or mid-journal with later as_of fields renumbered (K4) — makes mediated true. Insertion of a *witness* lowers standing; insertion of a *degrader* — a junk run by the witness's checker with a divergent replay and its cal_discredit, appended at the tail — raises it with no necessity root touched (the offline form of CF1). T11 catches all three: the stamp through its roots, the degrader through |Δ|, which counts *valid* witnesses.

(c) *Coherent rewrite is invisible to replay and preserves polarity.* If r' is a coherent rewrite of r then r' and r are replay-indistinguishable (T1), and every necessity of r that does not mention a rewritten root field holds of r'. A coherent rewrite of a *witness's* root fields — a run's verdict from refuted to survived together with its replay's, an adjudication's decision from accept to reject — removes the demonstration and is, for standing, a deletion: the witness clause of (a) applies to it.

Proof. (a) Note first that r' was not produced by the machine; the verifier merely holds it and computes []_B P_i on it as [[r']]_B ⊆ P_i, where [[r']]_B is the set of worlds whose history contains r'. So the comparison is between two world-sets defined by two records, not between two histories. By X1, every world consistent with r is consistent with its sub-record r': [[r]]_B ⊆ [[r']]_B, hence []_B P_i(r') ≤ []_B P_i(r) for *every* world property P_i — including properties defined by absence ("no refusal among the first n events"), which a pruned record can never certify, since the worlds consistent with it include those in which the pruned events happened. For the second clause: stand(r') > stand(r) with every necessity conjunct non-increasing forces some ¬d_{X_j} to have increased, i.e. d_{X_j}(r) ∧ ¬d_{X_j}(r'), i.e. a witness of shape X_j present in r, positioned consistently with μ, is absent from r'; for a validity-degraded demonstration d_X ∧ ¬d_Y the same holds because d_Y is itself monotone under sub-record. (b) [[r']]_B ⊆ [[r]]_B by X1, so a necessity can only rise; for a root-determined P_i it rises iff an inserted event supplies a (σ, ρ) that P_i names, which D26 lists among the positive atoms and T11 hashes. Inserting a witness adds to d_X; inserting a degrader voids a witness, which is T3's mixed case and is caught by the count of valid witnesses. (c) T1 gives indistinguishability; a root-determined necessity that does not name the rewritten field is evaluated on constraints unchanged by the rewrite; a witness whose verdict root is rewritten is no longer a word of shape X, so d_X falls exactly as under deletion — the witness clause of (a), not its inclusion (the logical branch's Probe 3a, reproduced: the rewritten escape replays clean as an audit and un-impeaches). []

What T4 does not say. It does not say deletion is detectable — it says what deletion must delete. (The first draft defined consistency as "produced exactly r", which contradicts X1 and leaves (a) without a proof; the reviewers were right, and D7 is the repair.) Nor does it let a record certify an *absence*: under D7 the custodial reading of "nothing else happened" is false at every record, which is the one-sentence form of the whole asymmetry — a record certifies presence, never absence — and the reason a negated demonstration cannot be a necessity. It does not say insertion is harmless — a forged *report* (a survived verdict from a lying harness) is an insertion whose root field violates B1, and B1 is assumed; that is the A10/B1 boundary the layer papers state, and here it appears as the boundary of [[·]]_B rather than as a caveat. It does not say coherent rewrite is harmless — (c) says exactly which necessities survive it: the ones that do not mention the rewritten root (a seal's artifact_hash mentions the sample's root, so a coherent rewrite of the artifact bytes' hash produces a seal for different bytes; §5 characterizes this).

Corollary T4.1 (The deletion surface). For a standing query stand and a record r, define $\Delta(r) = \{ e \in r$: e is an event of a witness $x \in X_j$ for some j with $d_{X_j}(r) \}$, the *deletion surface*. Then every deletion that raises stand removes at least one event of $\Delta(r)$, and $\Delta(r)$ is computable from r by the same re-derivation replay performs.

Not every event of $\Delta(r)$ is *deletable* by itself. An event is **anchored** if some later event's replay guard or recomputation reads what it contributed, so that deleting it alone leaves L_rep; it is **exposed** otherwise. Under the move set of §5 — in which recorded positions and run indices are roots a coherent alternative may renumber (T12c) — position fields anchor nothing, and the anchors are the *content* readers of rga/calibration.py:from_events: for an escape, a later cal_stamp of the same class whose cut the escape's validity precedes (its track_records and corpus_provenance are recomputed from the escapes valid as of sealed_at), a later cal_close(E5) of the same class naming a refuter the escape charges, when the demotion it records would not hold without that charge cell (demoted(·, as_of) is recomputed from the charges, one per cell however many witnesses), a later cal_run filed by the escape's own checker on a claim where it is not pinned, when this is that checker's only valid escape of the class at that point (_guard_audit_checker needs one), and a later cal_install whose ledger covers the class by a position-derived id (derived_defect_id reads the run's journal position) that this escape's deletion renumbers — deleting an earlier escape shifts a later covered run's id and _guard_install_covers no longer covers it — each reader evaluated as replay evaluates it, with the witness valid *at the reader*, so a witness established only after a stamp is not anchored by it; for a refusal group — the diverged rga_replay, the rga_refuse, and every rga_close(V4) the refusal cascaded onto open lines — a later same-class stamp whose cut the refusal precedes (refused_at ≤ sealed_at) in a class where the refused checker had a run valid at the stamp but for the refusal, since the refusal voids that run and the stamp's corpus differs (a refusal after the stamp's cut is invisible to it, _check_valid(as_of)), and a later cal_install whose cut the refusal precedes (refused_at ≤ as_of) when the refused checker had a run valid at the install but for the refusal, not excluded before it, that the installed policy does not cover — its class dropped, a bounded-only claim, or a ledger id-set omitting the run's derived id — since the refusal is then what emptied the obligation of D16, and the ratchet, recomputed on rebuild (_guard_install_covers, _guard_install_bounded), refuses the install once the run is back; a group no stamp and no install reads is exposed. A run the kernel replayed successfully more than once carries every establishing replay as a witness event; none is load-bearing alone, and the demonstration falls only when all are gone. A cal_install's coverage primaries are never recomputed on rebuild, but its ratchet is, and derived_defect_id reads a run's position: an install anchors an escape when deleting it renumbers a later covered run out of the ledger (above), and a refusal group when the refusal is what let the install pass — CF5 eases only when an escape vanishes without renumbering a survivor. Events that *name* a run — its replays, discredit and adjudication — are ties rather than anchors: they go with the run at no cost to any line (the companion's group_with). An exclusion naming the run is a *rewrite*, not a tie: the run is dropped from its run_indices, which empties and so removes one that named it alone and keeps the siblings of one that named others too, since deleted whole it would release those siblings into the obligation of a later install (rewrites); a rewrite is applied by the

re-derivation, not listed in deletion_closure. A sole establishing replay, and the accepting adjudication, take the run's exclusions too: deleting the sole witness leaves the run unestablished, so an exclusion that named it would name a non-valid escape on rebuild. Because derived_defect_id's position argument makes this enumeration hard to close by hand, the companion re-derives every candidate deletion (coherent): an event is reported exposed only when deleting its group and its rewrites and replaying is accepted, so a reader the analytic list missed still keeps the event off the exposed set; where several installs must go together — each covering the renumbered survivor only by its old id — the whole jointly necessary set is found by that re-derivation, not a per-install probe. Deleting an anchored witness alone is refused ("stamp does not recompute from the ledger", "E5 close without a demotion at that point"); deleting an exposed one replays clean. An anchored witness remains deletable *together with its anchors* — a stamp's deletion lowers its own line to IR, an audit carries its own structural group (its replays and adjudication, R4-15), and an install is read by whatever ran under the budgets it adopted, so a later E5 close of a class whose budget the install changed can join the closure (R4-13) — at the cost of the standing those anchors carried. The companion's deletion_closure builds that set and then *minimises it by re-derivation*: a candidate anchor or downstream reader is dropped whenever the alternative still rebuilds without deleting it — an E5 whose charge count clears the pre-install budget survives the install's deletion and is not listed (R4-16) — so the closure is exactly the deletion the kernel refuses to do without. This list was enumerated from the readers in from_events, not derived from the root-field dependency graph (that derivation is N11's job); the first draft of this paper said every demonstration was recompute-free, then that stamps and exclusions were the only anchors, the referees found the E5 close and the audit in an hour, and the automated review found the install a round later and, the round after, the position-derived id, the sole replay's exclusion and the install's own downstream close, and the pass after that an anchored audit's own replay and the reader an over-cautious closure did not need — the enumeration and its closure are now backed by a re-derivation of every candidate deletion, minimal and complete — the install probe runs alongside the analytic anchors, so an escape that needs both a same-class stamp and a renumbering install names both, and the alternative is renumbered consistently (a stamp shifted by a deletion carries its new position in track_records[*].as_of, T12c) — which is what closes it. tests/test_custody.py shows an escape after the last stamp exposed, the same escape anchored once a later same-class seal is stamped or an E5 close names its refuter, and a tail refusal group — with its cascaded close — exposed and deletable with a clean replay, a refusal after a same-class stamp's cut exposed while the escape before the stamp stays anchored, a run established by two replays carrying both as witnesses with neither exposed alone, a refusal group anchored by a later install whose ledger omits the escape it voided and exposed once the ledger covers it, a shared exclusion rewritten rather than deleted with one of its runs, a cal_run whose deletion renumbers a covered run anchored by the later install, a sole establishing replay deletable only once the exclusion that named its run is rewritten, an escape anchored by a replayed audit whose closure carries the audit's own replay, a run replayed twice whose closure for either replay names its sibling and the later same-class stamp that read the escape (deleting the demonstration, not one of two witnesses), and a refusal group followed by a mediated seal under a switched pin whose surviving stamp the closure re-binds by shifting its recorded admission position when the deleted group shortens the admission journal (the renumbering of T12c reaching across the two journals), and a tier-B run replayed twice whose closure for either replay carries its accepting adjudication, since removing every establishing replay leaves the run unestablished and the adjudication would otherwise be orphaned.

This corollary is a kernel capability the kernel does not have (§8, N1): the set of journal events whose deletion would raise any line's standing, as of a position, split into anchored and exposed. The composed paper's residue paragraph is a prose description of Δ; T4.1 says it is a pure query.

Corollary T4.2 (Why the receipt exists, and what it must cover). An anchoring scheme that certifies the produced record against coherent alternatives must pin, at minimum, (i) the root fields of every event a necessity mentions, against coherent rewrite; (ii) the presence of every event in $\Delta(r)$, against deletion; (iii) the length $|r|$, against truncation, since truncation is the deletion of a suffix and a suffix may

contain ∆-events. A hash-chained head over all events (the v0.5 receipt: docs/PROOFS_PLAIN.md, final section) pins all three. T4.2 is a sufficiency statement: (i) and (ii) suffice, and (iii) is what (ii) costs when it is implemented as a count rather than as the witnesses themselves (T11). It claims no minimality.

3.4 Loudness as a theorem about attempts

The composed paper's Theorem 2 ("Loudness of deviation") is proved by exhaustion of a fault table. Under D3 it is a structural statement.

T5 (Loudness). Let stand be a standing query whose necessity conjuncts are each the conclusion of a transition guard (the seal's guard for is_sealed, the stamp's binding for mediated). Then every attempt that would, if it succeeded, make a necessity conjunct true without the guard's premises is refused (no event) or publishes (no store write precedes a fault event), and no attempt removes a demonstration — the interface is append-only. An attempt *can* make a negated, validity-degraded demonstration conjunct true by degrading the witness's validity: replay_run with a divergent report discredits the checker (rga/calibration.py:replay_run), Admission.replay with a divergent report refuses it, and _check_valid then voids every run of that checker, established or not. These are the only standing-raising attempts outside the seal's own guard chain, and each is triggered by a *report* (B1), not by a re-derivable fact. §8a, CF1, is what that costs.

Proof. The first clause is the definition of a guarded fold with the refuse/publish partition (D2, D3): the only way to append the event that makes the conjunct true is through its guard. The second clause: no attempt in Att deletes events, so d_X is non-decreasing along attempts (T3); validity degradation is _check_valid read literally — its discredited and refused conjuncts are set by the two replay transitions and by nothing else. []

What T5 does not say. T5 is about attempts through the interface; it says nothing about an adversary who edits the journal file, which is T4 and §5. It also does not say that the fault table is *complete* — that every falsifying sequence maps to a named fault. Completeness is the adequacy question of §6.2, and the composed paper is right to list F2 as unproved.

3.5 Summary of the semantics

A custody is a record system (Σ, μ) with a replay language L_rep, a world semantics (W, |=, B) yielding the custodial reading []_B, and a family of demonstration shapes X_j. Its standing queries are conjunctions ∧[]_B P_i ∧ ∧¬d_{X_j}. Necessities are re-derivable and monotone; demonstrations are witnessed and monotone; standing is neither, and its non-monotonicity is exactly located in its negated demonstrations. Deletion attacks demonstrations; coherent rewrite attacks roots; truncation attacks both; insertion attacks the positive atoms replay checks but does not question (a stamp for an unmediated seal) and the validity of a demonstration, offline by an appended degrader and through the interface by a report the kernel accepts as its falsifier (T5, CF1). That is the whole of Admissible's architecture in one paragraph, and the next four sections are what each facet of it costs and buys.

3.6 The support, and the one theorem the facets share

D26 (Signed support). For a standing query stand and record r, the *support* supp(stand, r) is the pair (roots(stand, r) ∪ degraders(stand, r), ∆(r)): the *positive atoms* are the root fields, with their positions, that the necessity conjuncts mention, together with the events that void a witness against the line (a cal_adjudicate(reject) is not one: the tier-B accept is a positive conjunct inside the witness, and deleting a rejection leaves the run unadjudicated and still invalid); the *negative atoms* are the surface — the events of every valid witness, including the closes a refusal cascaded — each marked anchored or

exposed, and the two halves may overlap.

T17 (Support representation). For every standing query on L_rep: (i) *determination* — stand(r) depends on r only through supp(stand, r), where the positive atoms include, besides the necessity roots, the *validity degraders* of every voided witness against the line (the diverged replay and discredit of its checker; the refusal group of its checker — which may also be a taint witness of the same line, so the two halves of the support can overlap): removing any event outside the support leaves the value unchanged, provided the result lies in L_rep, while removing a degrader revives a witness and lowers it; (ii) *fail-closed is infimum* — each necessity conjunct is the infimum of P_i over [[r]]_B, a function of the positive atoms alone, and each negated demonstration is the infimum over the presence of its witnesses, a function of the negative atoms alone; (iii) *anchoring* — a hash of the necessity roots, a count of the negative atoms and a length witness is an anchor (T11); every negative atom, anchored or not, must be counted, since anchoring prices a deletion and does not prevent it (T10b); (iv) *readability* — a negative atom is *committed* or *selected* in the sense of D28 (a committed observation ran at a seed from a nonce fixed before the run and unpredictable to whoever chose the bytes, with a filing decision independent of the outcome); today every negative atom of admissible is selected — a filer chooses what to file — and only committed observations license the inferences of §4.6.

Proof. (i) roots, the degraders and Δ are read off the guard structure and _check_valid; the companion checks both halves — removing every event of a later, unrelated line leaves admissible unchanged, and removing the discredit pair that voided an escape revives it (tests/test_custody.py, SupportDetermination, SupportIncludesValidityDegraders). (ii) is D8 and D9. (iii) is T11 with T10(b). (iv) is D28 restated. []

The support is the object the four facets of §4–§7 project: §4 restricts it to defect ids and kill records; §5 asks which atoms a coherent alternative can move; §6 asks which guard refuses each atom's absence; §7 asks which atoms the generator's information set contains. The completeness critic of this paper's own review called the object a *committed support presheaf*; the name here is shorter and the claim smaller — one signed, computable set per query (custody.py:support) — and the literature name for its positive half is de Kleer's ATMS label (1986), for its determination clause Cheney–Chiticariu–Tan's where-provenance (2009). What is not in either is the sign, and the sign is what §5 and §8a are about.

4. The algebra of carried doubt

The scrutiny layer's second contribution is "measured doubt as a carried record": power enters a seal only as a kernel-counted primary, composes only by union or max, and is never a product. The layer paper defends each clause by adversarial review. This section shows that the three clauses are one theorem — the kernel is computing the Fréchet–Hoeffding bounds of its own primaries — and then says what the theorem licenses that the kernel does not yet do.

4.1 Objects

D12 (Committed defect model, kill record). A *committed defect model* is a finite set D whose identity (a content hash) fixes its element set on first record (rga/core.py:Admission.measure, defect_ids.setdefault; V13). A *kill record* for refuter ρ on D is a subset $K_{\rho,D} \subseteq D$ (PowerRecord.killed_ids). Its *density* is $|K_{\rho,D}| / |D|$ (PowerRecord.power, ledger mode). A density is a primary: it is exact on D and asserts nothing about anything else.

D13 (Bounded record). A *bounded record* is a declared pair (ε, N) with carried figure $1 - (1-\varepsilon)^N$ (rga/core.py:Admission.bound). The figure is one minus the probability that N independent trials, each succeeding with probability at least ε, all fail — the probability that at least one succeeds, a lower bound — and it therefore carries, inside itself, an independence assumption across the N draws that is not

among B0–B14. The kernel labels this by mode on the seal (R2, "the figure is as declared as ε is"). In the vocabulary of §4.3, a bounded record is the kernel's one *coupling certificate*, and it is declared rather than measured.

D14 (Reading). A *within-model reading* of a kill record is the choice of a probability distribution on D; under the *uniform reading* U_D, density is P_{U_D}(defect ∈ K_{ρ,D}), the probability that a defect drawn uniformly from D is caught. The kernel never chooses a reading (B6: "the reading is the reader's, not the kernel's"). Every statement in this section of the form "the probability that …" is under a reading the reader has chosen, and the theorems are about which readings' consequences are sound *under every coupling*.

4.2 Composition across refuters on one claim

K2 (What the kernel computes). For a claim j with defect model D_j, ledger refuters L with kill records K_ρ ⊆ D_j, and bounded refuters b with figures β_b (rga/core.py:Admission._claim_seal): u_j = |∪_{ρ∈L} K_ρ| / |D_j|, composite_j = max(u_j, max_{b∈b} β_b), labelled union if only L contributed with |L| ≥ 2, single if one refuter, max otherwise; and power_min = min_j composite_j (Admission.seal).

T6 (Union is exact; max is the Fréchet lower bound; the product is unsound). Fix a claim j and a reading U_{D_j}.

(a) Over a shared committed D_j, u_j is exactly P(defect ∈ ∪_ρ K_ρ): there is no coupling freedom, because the extents are known sets in one space.

(b) Across records on different spaces (a ledger record on D_j and a bounded record, or, in a generalization the kernel does not implement, records on D_j and D'_j), let A_ρ be the event "ρ catches the defect" with P(A_ρ) = p_ρ under its own reading. For every coupling C of these events, P_C(∪_ρ A_ρ) ≥ max_ρ p_ρ, and the bound is attained. Hence max is the greatest aggregator sound under every coupling, and any aggregator g with g(p) > max(p) for some p is unsound for some coupling.

(c) In particular 1 − ∏_ρ (1 − p_ρ) (noisy-OR, the product rule) exceeds max whenever two p_ρ are positive and none is 1, and is attained under the independent coupling; it is sound only under a certified coupling with P_C(∩ A_ρ) = ∏(1−p_ρ), of which independence is the one the kernel could name.

Proof. (a) |∪ K_ρ| / |D_j| = P_{U}(∪ K_ρ) by definition of the uniform reading. (b) For any coupling, ∪_ρ A_ρ ⊇ A_ρ for each ρ, so P_C(∪ A_ρ) ≥ p_ρ; take the max. Attainment: the comonotone coupling — realize each A_ρ as the initial segment of length p_ρ of one uniform variable on [0,1] — gives ∪ A_ρ = the longest segment, of measure max p_ρ. Under uniform readings on finite sets with rational densities, the same coupling exists on a finite common refinement (a circle of lcm |D_ρ| points), so attainment is not an artefact of continuity. That max is the greatest sound aggregator: if g(p) > max(p), the comonotone coupling has P_C(∪ A_ρ) = max(p) < g(p). (c) 1 − ∏(1−p_ρ) ≥ max p_ρ, with equality iff at most one p_ρ > 0 or some p_ρ = 1 (a boundary bound(ε = 1, N = 1) reaches, CF6). Since P_C(∪A_ρ) = 1 − P_C(∩ A_ρ), equality with noisy-OR holds iff P_C(∩ A_ρ) = ∏(1−p_ρ): the product coupling satisfies this, and for three or more events it is not the only coupling that does. []

What T6 does not say. It does not say max is a *good* estimate of anything; it says max is the only figure that is not a lie under some coupling. It does not license reading composite_j as the power against real defects: that is B6, unchanged.

T6.1 (Why the kernel's labels are the right labels). The three composition labels the seal carries — single, union, max — are exactly the three cases of T6: one record (no composition), several records on one space (exact by (a)), several records on different spaces (Fréchet by (b)). The label is the

coupling regime, and the seal is telling the reader which theorem it used.

4.3 Composition across claims and along dependency edges

The seal also carries power_min, the minimum of the claim composites, and the dependency gate check_dependencies(deps, floor) requires each dependency's power_min ≥ floor (rga/core.py:Admission.check_dependencies; rga/calibration.py:CalibrationAuthority.check_dependencies). Both are minima. Under the custodial reading a minimum is sound as a statement about *each* conjunct; read as a statement about *all* conjuncts jointly, it has the opposite polarity.

T7 (Conjunction: min is the upper bound; Bonferroni is the lower). Let A_1, …, A_n be events "the defect of claim/dependency i is caught" with P(A_i) = p_i under their readings. For every coupling C, $\max(0, 1 - \Sigma_i (1 - p_i)) \le P_C(\cap_i A_i) \le \min_i p_i$, both bounds attained. Hence the greatest aggregator sound for the joint reading "every conjunct caught" is $\mathrm{bon}(p) = \max(0, 1 - \Sigma(1-p_i))$, and min is the *least aggregator that is never exceeded* — the fail-open direction.

Proof. Upper: $\cap A_i \subseteq A_i$. Lower: Boole's inequality on the complements, $P(\cup A_i) \le \Sigma(1 - p_i)$. Attainment of the lower bound: lay the complements A_i end to end on the circle of circumference 1; they are disjoint iff $\Sigma(1-p_i) \le 1$, in which case $P(\cap A_i) = 1 - \Sigma(1-p_i)$; otherwise they cover the circle and $P(\cap A_i) = 0$. Finite uniform readings: as in T6(b). []

Corollary T7.1 (The Bonferroni horizon). Under the joint reading, n conjuncts each at power p carry assumption-free joint assurance $\max(0, 1 - n(1-p))$, which is zero at $n \ge 1/(1-p)$. At p = 0.9 the horizon is ten; at the bench's carried powers (eval/bench/RESULTS.md) it is computable per seal and per dependency closure.

Corollary T7.2 (Folds on a DAG). The sharp assumption-free deficit of a line and its ancestors is the *ideal sum* $\Sigma_{u \in \mathrm{Anc}^*(v)} \Sigma_j (1 - \mathrm{composite}_j(u))$, each ancestor counted once. A min-plus fold along paths returns the smallest single-path deficit, which understates the ideal sum on any DAG with two paths to a shared ancestor and is therefore unsound; a sum over all paths counts a shared ancestor once per path and is sound but not sharp. The companion's power_joint_closure is the ideal sum over every claim of every ancestor — not 1 − power_min per node, since power_min is itself a minimum and would import K3's polarity flip inside each node.

K3 (Reading power_min correctly). power_min is exact and sound under the per-claim reading ("no claim was scrutinized below power_min"), which is what R2 and V5 assert. Under the joint reading it is the Fréchet upper bound of T7: a consumer who reads power_min = 0.9 on a three-claim seal as "this artifact was scrutinized at 0.9" has read an upper bound as a lower bound; the assumption-free joint figure is 0.7. The kernel does not make the joint reading; nothing here says it should. It says the joint reading has a sound value, the kernel can carry it as a primary of primaries (§8, N4), and the dependency gate's per-edge floor is, transitively, a per-edge statement whose joint value along a chain of n edges at floor f is at most $\max(0, 1 - n(1-f))$ — and exactly the ideal sum of T7.2, $\max(0, 1 - \Sigma_i m_i(1-f))$ over the m_i claims of each dependency, since a floor on power_min is a floor on every claim — not f.

What T7 does not say. It does not say the Bonferroni figure is the artifact's probability of being correct. All three quantities are within-model readings of instrument coverage (B6). It does not say the kernel is wrong to gate on the minimum: gating each conjunct at a floor is exactly right for a per-conjunct guarantee. It says only that the *name* power_min invites a reading whose sound value is smaller, and that the sound value is computable from the seal alone.

4.4 The kill context

A kill record is a set; the seal reports its size. The structure the kernel already stores and does not report is the incidence.

D15 (Kill context). For a claim with committed model D and ledger refuters R_D, the *kill context* is the formal context (D, R_D, I) with d I ρ ⇔ d ∈ K_{ρ,D}. Its *extents* are the kill records; its *uncovered set* is D \ ∪_ρ K_ρ; the *unique kills* of ρ are K_ρ \ ∪_{ρ'≠ρ} K_{ρ'}; its *concept lattice* is the Galois lattice of the incidence (Ganter–Wille).

T8 (The composite is the union's density; redundancy is a set fact). u_j equals the density of the union of the kill records, |∪ K_ρ|/|D|. A refuter ρ is *redundant on D* — its removal from the claim leaves u_j unchanged — iff its unique-kill set is empty, K_ρ ⊆ ∪_{ρ'≠ρ} K_{ρ'}. The union is in general not an extent of the concept lattice of D15 (the top concept's extent is all of D), and lying below the join of the others' attribute concepts does not imply redundancy, since the join closes the union: the lattice is the kernel's record of the incidence, and the composite and the redundancy are set facts about it.

Proof. Immediate from D15 and K2: removing ρ changes ∪ K_ρ iff K_ρ ⊄ ∪_{ρ'≠ρ} K_{ρ'}. []

What T8 does not say. Redundancy on D is not redundancy against real defects; a refuter with no unique kills on D may be the only one that reaches a defect class D does not contain. That is the coupling hypothesis again, and it is why T8 is a fact to *carry* (§8, N5: uncovered size and unique-kill counts as seal primaries) rather than a reason to drop a refuter.

4.5 Coverage as a Galois connection

The ratchet (C4) says a successor policy installs only if its ledger defect models cover the class's valid escape corpus minus journaled exclusions (rga/calibration.py:_guard_install_covers). The layer paper's own gap inventory notes that the coverage relation is structurally undefined. It is not: it is id-set containment, and that choice is forced.

D16 (Obligation, coverage). For a class c at position t, the *obligation* is Ob_c(t) = { derived_defect_id(e) : e ∈ corpus(c, t) } (rga/calibration.py:derived_defect_id, corpus); it is the valid escape corpus minus exclusions. An element leaves it in two ways: through a journaled, attributed exclusion (exclude, E7), or because its witness is voided — its checker discredited (cal_discredit) or refused (rga_refuse), the ± of D6 and the CF1 shape — and only the first is an exclusion. For an admission policy π, the *coverage* of c under π is Cov_c(π) = ∪_{claim ∈ π(c)} ids(D_claim) (adm.defect_ids). π is *installable at t* iff Ob_c(t) ⊆ Cov_c(π) for every c that owes coverage, and no owing class is dropped.

T9 (The ratchet is a closure condition; installable policies form an up-set; membership is the weakest decidable non-discretionary coverage). (a) t |-> Ob_c(t) is non-decreasing except through exclude and through the voiding of a witness (D16). (b) The set of installable policies at t is an up-set in the coverage order (π ≤ π' iff Cov_c(π) ⊆ Cov_c(π') for all c). (c) Any coverage relation covers(D, e) that is decidable by the kernel from journal state alone and non-discretionary (needs no adjudication event) is a function of derived_defect_id(e) and ids(D) alone, since the kernel holds nothing else about either; among such relations that are monotone in the id-set and invariant under renamings of ids that fix derived_defect_id(e), membership is the weakest that distinguishes a covered escape from an uncovered one at every id-set, and it is the one the kernel implements. Anything that distinguishes two escapes with the same journal identity, or two models with the same id-set, is not kernel-decidable.

Proof. (a) corpus grows with escapes, which grows with established valid runs (T3), and shrinks only via exclusions or via _check_valid when a run's checker is discredited or refused (D16). (b) If Ob_c ⊆

Cov_c(π) and Cov_c(π) ⊆ Cov_c(π') then Ob_c ⊆ Cov_c(π'). (c) The kernel's state about an escape is a Run whose only defect-identifying content is its journal identity (derived_defect_id is a pure function of line_id, claim_id, position); its state about D is defect_ids[D]. A relation decidable from these and requiring no further event is a relation between an id and an id-set; monotone and renaming-invariant ones are generated by membership and by thresholds on |ids(D)|, and a threshold distinguishes no two escapes at any id-set (at a singleton it distinguishes nothing at all), so membership is the weakest that does at every id-set. Any semantically richer coverage — "D contains a defect of the same class as e" — requires a judgment the kernel cannot make, i.e. a tier-B adjudication (E2), and is therefore discretionary. []

What T9 does not say. It does not say id-containment is a *good* notion of coverage against real defects; it says it is the weakest non-discretionary one that does the ratchet's work, that the first draft's "only" was too strong (a threshold on |D| is another, and useless), and that richer coverage is exactly the tier-B channel the kernel already has. The gap the inventory names is real, and T9 locates it precisely: semantic coverage is an adjudicated obligation, not a kernel one.

4.6 Commitment and readability

The layer papers say that silence in the escape corpus is unreadable and that zero charges is absence of filed evidence. Under a committed/selected split that sentence is a theorem, and it has a converse that says what *is* readable.

D27 (Seed-mass). For a sealed artifact a, a pinned refuter ρ and a claim, ρ(ρ, a) = Pr_s[ρ(a, s) = refuted] over seeds s = H(ν | h_a | ρ | claim) with ν uniform — a property of one deterministic function under B2, not of any population.

D28 (Committed, selected). An observation of ρ on a is *committed* if its seed is the kernel's derivation from a nonce fixed before the run and unpredictable to whoever chose the bytes (B9 read as unpredictability, not only as ordering), and the decision to file it did not depend on its outcome; *selected* otherwise. Seal trials are committed (nonce drawn after every guard, rga/core.py:sample; seed forced, _guard_trial_seed; one per cell). A filed escape or audit is selected: _file_run accepts any nonce, and the finder files what it chooses.

T18 (Selection identifies only existentials; commitment licenses an e-value). (a) For every filing strategy, the set of values of ρ(ρ, a) consistent with any finite collection of selected filings is (0, 1) if it contains an established escape and [0, 1) otherwise: the seal's own committed surviving trial certifies ρ < 1, an established escape certifies ρ > 0, and no number of selected audits moves either endpoint. (b) If a *campaign* — a nonce family of declared size N, drawn by the authority's unpredictable source and journaled before any run — is run to completion with every run filed, then under B2 and B9-unpredictability E_N = (1 − ρ_0)^{−N} · 1[all N survived] has expectation at most 1 under the hypothesis ρ(ρ, a) ≥ ρ_0: an e-value, valid under optional stopping across campaigns by Ville's inequality. An incomplete campaign is valued 0, and the product over *all* campaigns opened on (line, claim, checker) is again an e-value; a product over completed campaigns only is not, since a finder who runs many privately and completes the survivors has selected again.

Proof. (a) A selected filing is a point (s, verdict) of a function the filer evaluated wherever it liked; a finite set of such points constrains ρ only through ρ > 0 (a refuted point) and ρ < 1 (a survived point), and the seal supplies the latter. (b) Under B9-unpredictability the N seeds are independent of a and uniform, so Pr[all survive] = (1 − ρ)^N ≤ (1 − ρ_0)^N under the hypothesis; the incomplete-campaign valuation removes the second selection; Ville gives the stopping rule. []

What T18 does not say. It does not say ρ is the generator's defect rate; it is the seed-mass of one instrument on one artifact, and B6 stands twice over. It rests on B9 read as unpredictability, which the kernel enforces by ordering only, and which neither the test harness's nonces (nonce-{i}, tests/test_rga_invariants.py) nor the bench's (n{i} for samples, finder-{iid}-{j} for escapes, eval/bench/bench.py) satisfy — a labelled assumption, not a journal fact; and on the seeds H(ν | …) being uniform for uniform ν, a pseudorandomness property of H that B7's second-preimage resistance does not supply and that must be named beside it. The e-value and its stopping rule are Ville (1939), Shafer (2021), Vovk & Wang (2021) and Grünwald, de Heide & Koolen (2024); the zero-valued incomplete campaign is the standard e-process device. The zero-failure bound in (b) is Hamlet (1987) and Miller et al. (1992); the commit-then-sample architecture is that of risk-limiting audits (Stark 2008; Waudby-Smith, Stark & Ramdas 2021). What is local is the mapping of derive_seed and _file_run onto those roles, and the fail-closed valuation of an unfinished campaign (§8, N23).

K10 (A charge is a co-liability). charge_cells adds a cell for *every* refuter pinned on a claim whenever any valid escape stands on that claim — including a tier-B escape by a checker pinned nowhere — and the bench's spec-weak carries charges earned by spec-strong's escapes at spec-strong's seeds (eval/bench/RESULTS.md). A charge therefore certifies that the refuter's committed survived was wrong on an artifact some instrument refutes, not that the refuter's own seed-mass is positive; only the tier-A escape's own checker receives the certificate of T18(a). C2 is exactly right about what it counts, and this is what it counts.

5. The record under rewriting

The composed paper's Theorem 2 ends with a paragraph of residue: coherent rewrite of root inputs, tail truncation, recompute-free deletion, and endpoint stamp forgery are "indistinguishable from another honest history to replay alone". Under D11 these are moves on L_rep. This section characterizes the moves, the invariants of their action, and what an anchor must and need not pin.

5.1 The rewrite groupoid

D17 (Moves). On L_rep define four families of partial maps:

• *truncation* τ_n(r) = r[0:n] for n < |r| — total, by prefix-closure;

• *root rewrite* ρ_{e,f}(r): replace root field f of event e and recompute every derived field of e and of every later event whose derivation consumes f, defined iff every guard along the way still holds;

• *recompute-free deletion* δ_G(r): remove a set G of events, defined iff no surviving event's derivation or guard consumed a field of any event in G and the result re-drives;

• *endpoint insertion* ι_e(r) = r·e, defined iff r·e ∈ L_rep. The *rewrite groupoid* G is the groupoid generated by these moves and their inverses where defined; the *orbit* G·r is the set of coherent alternatives to r.

K4 (The kernel's residue list is a generating set). The four families are, verbatim, the composed paper's residue: root rewrite ("a coherent rewrite of root transition inputs"), truncation ("a journal shortened at the tail"), recompute-free deletion ("a coherent removed group no surviving event recomputes against"), and insertion ("a calibration stamp forged for an unmediated seal at that seal's own position in stamp order, later as_of fields renumbered" — the record branch's E1, reproduced). The kernel's from_events refuses everything *else* — inconsistent derived fields, forged transitions, duplicates, and deletions that a later event recomputes against — because each of those leaves L_rep (T1).

T10 (What replay leaves to the anchor). The groupoid G is connected — τ_0 carries every record to the empty one and endpoint insertions carry it back — so no non-constant query is invariant under all of G, and "certifiable by replay alone" is empty. Two refinements make the statement useful. First, a *length witness*: let G_n be the subgroupoid generated by the length-preserving *atomic* moves — a root rewrite with the recomputation of every later derived field, a consistent renumbering of position and index roots, and ι_{e'} · δ_e for a single event e whose deletion alone leaves L_rep (deleting it and inserting one event anywhere, with the same recomputation). Second, two notions of certifiability: a query is *value-certifiable* under a length witness iff its value is constant on G_n-orbits (D18); a necessity is *proposition-certifiable* iff, in addition, the root constraints of [[r]]_B it speaks of are constant — the same bytes, the same nonces, the same refuter versions. Then: (a) no necessity of the kernel is proposition-certifiable under a length witness: each mentions a root a rewrite can move while the boolean stays — a root rewrite of the sealed artifact's hash leaves is_sealed(id) true of *different bytes*; the boolean is value-certifiable, and it is the proposition a consumer asks about; (b) no witness-bearing negated demonstration is value-certifiable under any length-preserving move set that admits verdict rewrites or single-event delete-and-insert pairs: an exposed witness is removed by one such pair; an anchored one by peeling its anchors first, each of which is itself exposed (a stamp, an E5 close, an audit); and a witness's verdict or decision roots are rewritten by another move, which replay accepts. *Anchored* and *exposed* therefore classify the **cost** of a deletion — which other lines it lowers — and not certifiability; the anchor must carry the count of valid witnesses in full, which T11 does; (c) hence neither value of admissible(id) is certifiable under a length witness without the anchor of T11: when true, its proposition is not (a); when false by a witness, an exposed witness flips it (b); when false by ¬mediated, the insertion of a stamp for the unmediated seal, at the endpoint or mid-journal with later as_of fields renumbered, paired with the deletion of any exposed event, flips it — the kernel's K4 residue, reproduced. A record with no exposed event to pair with — a lone IR seal — is invariant under G_n by exhaustion of moves, not by structure.

Proof. Connectedness: τ_0 then endpoint insertions. (a) A root rewrite changes the constraint the root imposes and no derived field; the value of a necessity that reads roots only through the guards they satisfied is unchanged, its proposition is not. (b) The companion exhibits both the clean replay of an exposed escape's deletion and the refusal of an anchored one; the verdict rewrite is the logical branch's Probe 3a, recorded in REVIEWS.md. (c) From (a) and (b), and for the stamp insertion the record branch's E1 and round 2's P5/P16 (REVIEWS.md), which the roots component of T11 catches. []

What T10 does not say. It does not say replay is useless: replay refuses every alternative outside L_rep — every inconsistent tampering — and every deletion of an anchored witness by itself. It says what the anchor must pin: the roots, every valid witness, and the positive atoms an insertion can supply. That is T11.

5.2 What an anchor must pin

D18 (Anchor). An *anchor for q at t* is a function α from records to a finite certificate such that, for all r, r' ∈ L_rep with |r| ≤ t, α(r) = α(r') implies q(r) = q(r'). An anchor is *global* if α is injective on L_rep up to t (a hash chain of every event — the v0.5 receipt's three heads), and *scoped* otherwise.

T11 (Scoped anchor for a standing query). Let stand = ∧ []_B P_i ∧ ∧ ¬d_{X_j} and let roots(stand, r) be the multiset of root fields, with their event positions, that the P_i mention. Then α(r) = (H(roots(stand, r)), |Δ_stand(r)|, |r|) is an anchor for stand at every t ≥ |r|. It is smaller than the global anchor whenever stand mentions a proper subset of events, and for stand = admissible(id) it is a *per-line* certificate: the roots of that line's open, stage, bind, call, decide, accept, rga_open, rga_sample, rga_trial, rga_replay, rga_seal, cal_stamp events and of the rga_declare/rga_measure/rga_bound records of every refuter its seal pinned, the count of surface events against id (the events of every valid

escape and of every tainting refusal group), and the journal length.

Proof. Take r, r' with $\alpha(r) = \alpha(r')$. The necessities: $[[\cdot]]_B \subseteq P_i$ depends on the record only through the roots P_i mentions and the guard structure, which is fixed by L_rep; equal root multisets (with positions) give equal necessities. The negated demonstrations: $|\Delta(r)| = |\Delta(r')|$ with $|r| = |r'|$ — could r' have deleted a witness against id and inserted a different witness against id? Then d_{X_j} holds of both and $\neg d_{X_j}$ agrees. Could r' have deleted a witness and inserted an unrelated event to preserve length? Then $|\Delta(r')| < |\Delta(r)|$, contradiction. Could r' add a witness (lowering standing)? Then $|\Delta(r')| > |\Delta(r)|$, contradiction — and a lowering is the fail-closed direction in any case. Hence stand(r) = stand(r'). Size: the global anchor hashes every event; α hashes the roots stand mentions and counts the rest — it still moves when an event is inserted *before* a necessity event, since positions are hashed, which is a refusal of an honest re-ordering in the fail-closed direction. []

What T11 does not say. The certificate must be *signed* by the authority that held the journal at t, and the signer's key, the registry it is published to, and the first anchor's own past are B14 and the composed paper's "unauthenticated-history residue", unchanged. T11 changes what is signed, not who is trusted. It also does not say the count $|\Delta|$ is enough to *identify* which escapes stand — a consumer who wants to know *which* defects were found needs the witnesses, not their count; T11 pins the *standing*, which is all admissible returns.

K5 (What the v0.5 receipt pins, read through T11). The composed receipt binds three full heads and the I/R/C predicates (tests/test_authenticated_receipt.py). That is a global anchor: sufficient by T4.2, and more than T11 requires for any single line. T11 licenses a *line-scoped* standing certificate (§8, N2) that a consumer can hold per dependency: in shape an OCSP response (RFC 2560 — the layer papers' own ancestor for impeachment), a status for one subject at one time, signed. Two such certificates compose along depends_on only if they speak of the same journal, which the $|r|$ component alone does not establish (two journals of one length); composition therefore re-imports a journal identity — a namespace and the hash of the length-n prefix, or the registry head — exactly as two Signed Tree Heads of one log compose only through a consistency proof (Crosby & Wallach 2009; RFC 6962 §2.1.2). What the scoped certificate saves is not the identity but the proof: given the identity, two of them compose by equality of $|r|$.

5.3 Position and preimage

Every temporal guard in the kernel witnesses an ordering. There are two ways to witness one, and they defend against different adversaries.

D19 (Position-witnessed, preimage-witnessed precedence). For events e, e' in a record, $e \lessdot e'$ is *position-witnessed* if the guard that requires it reads journal positions (fcd_position recorded at rga_open and rga_sample, refused_at ≥ sealed_at in tainted, membership of a refuter in refuters at Open standing for declared_at < opened_at). It is *preimage-witnessed* if e' carries a field equal to H(x) where x contains a root value that first exists in e — a nonce drawn at e, or the hash of e's own root fields — and H is second-preimage resistant (B7).

T12 (Two witnesses, two adversaries). (a) A preimage-witnessed precedence cannot be produced by the *online* adversary (a party acting through Att who cannot invert H) in the wrong order: to write e' one must already hold x, hence e already exists. (b) A preimage-witnessed precedence is invariant under every move of G that preserves e's root values, and a root rewrite of e propagates into e''s derived field, so the *dependency* survives rewriting; what rewriting cannot do is produce an e' whose derived hash has a preimage the rewriter did not choose. (c) A position-witnessed precedence is invariant under no move that renumbers: root rewrites and deletions elsewhere in the journal shift positions, and a coherent rewrite that keeps all guards satisfied can make a position-witnessed guard hold of an order in

which it did not. (d) A position-witnessed precedence *is* unforgeable online, because the kernel reads its own position (rga/core.py:Admission.open "a caller-supplied position would bypass the before-generation guard").

Proof. (a) is the definition of second-preimage resistance applied to a root first created at e. (b) Derived fields recompute from roots; the dependency e' |> e is a fact about the derivation, which G preserves by definition of L_rep. (c) Positions are not fields of events; they are indices in the word, and a guard that consults a *recorded* position consults a root. The recorded fcd_position at rga_open is such a root: on the live path the kernel reads its own position (rga/core.py:Admission.open), but on replay the value comes from the journal and is checked only for range (Admission.from_events: 0 <= ev["fcd_position"] <= fcd._position()). A coherent rewrite that lowers it to a value preceding the sample stages satisfies _guard_open_before_generation on replay whatever the true order was — every derived field is unchanged, so the rewritten record is in L_rep. The same holds of the fcd_position recorded at rga_sample for _guard_sample_order. (d) The kernel does not accept a position from the caller on the live path. []

K6 (Classifying the kernel's temporal guards). Preimage-witnessed: R10 (seed = H(ν|h|ρ|claim), rga/core.py:derive_seed); the tier-A escape seed (rga/calibration.py:_guard_run_seed); the rga_seal event carries no position field — sealed_at lives only in the Seal dataclass — so the scrutiny journal has no self-anchor of its own; the standing journal's cal_stamp.sealed_at is compared on replay against the *rebuilt* seal's position (rga/calibration.py:from_events), which makes every mediated seal an anchor for the scrutiny journal's length below it and leaves an unmediated (IR) seal unanchored. mediated is therefore mixed: a position root bound by content. Two further facts, both reproduced: as_of restricts only the scrutiny axis — it bounds the refusal clause of _check_valid and nothing on the standing axis, so a retrospective track_record(as_of_seal) differs from the stamped value once later escapes exist; and CalibrationAuthority.open emits no event, so C6's open-time demotion guard is never re-verified on rebuild. Position-witnessed: V8's before-generation half (_guard_open_before_generation), V8's interleaving half (_guard_sample_order), tainted's refused_at ≥ sealed_at, _check_valid's refused_at ≤ as_of. By T12(c), the position-witnessed ones are the guards whose *offline* forgery the residue paragraph describes; by T12(a),(d) all of them are sound online. §8 (N7) proposes preimage witnesses for the two V8 halves, the guards on which R3's separation of duty rests.

What T12 does not say. It does not say a preimage witness proves *time*: it proves dependency, and dependency is order only for a party who cannot invert H. It does not make an offline rewriter unable to rewrite both e and e' coherently; it makes such a rewrite change the roots the anchor of T11 pins.

6. Stacked custody

The kernel is three machines composed by "disjoint write sets and read-only observation", with two non-interference lemmas (R11, C7) each proved by inspecting every write in the upper machine. This section states the composition once, so that the lemmas are instances and a fourth machine composes by the same theorem; and it states the adequacy question the delete-the-guard methodology answers by exhaustion.

6.1 Projection and interposition

D20 (Footprint). A machine M = (Σ_M, St_M, μ_M) has a *write footprint* Σ_M (the event types it appends) and a *read footprint* Rd_M ⊆ St (the state components its guards and steps consult). Two machines M1, M2 are *write-disjoint* if Σ_{M1} ∩ Σ_{M2} = ∅.

D21 (Stacking). M2 is *stacked over* M1, written M1 « M2, if (i) they are write-disjoint; (ii) M2's guards and steps may read μ1's state but write only St_{M2} (read-only observation); (iii) whenever M2 needs

an M1-transition to occur, it invokes M1's own attempt and does not construct the event (guarded interposition). The *combined machine* M1 ⊕ M2 folds words over Σ_{M1} ∪ Σ_{M2}, dispatching each event to its owner.

D22 (Projection). For M1 « M2, the *projection* π1 : (Σ_{M1} ∪ Σ_{M2}) → Σ_*{M1}* erases M2's events.

T13 (Invariant inheritance). If M1 « M2 then (a) π1 maps L(M1 ⊕ M2) into L(M1) and μ_{M1}(π1(r)) = (μ_{M1⊕M2}(r))|_{St_{M1}} — the combined machine's M1-state is the M1-machine's state on the projected word; (b) every invariant of M1 — every property Φ of St_{M1} such that Φ(μ_{M1}(r)) for all r ∈ L(M1) — holds of the M1-component of every reachable state of M1 ⊕ M2; (c) every *history* invariant of M1 — a property of words in L(M1) — holds of π1(r) for every r ∈ L(M1 ⊕ M2).

Proof. (a) By induction on r. An M2-event changes no St_{M1} component (D21 ii) and is erased by π1, so both sides are unchanged. An M1-event in the combined machine was appended by M1's own attempt (D21 iii) under M1's own guard evaluated on the St_{M1} component, which by induction equals μ_{M1}(π1(r)); so the guard holds in M1 on π1(r), and both sides step identically. (b) and (c) follow from (a): the M1-component of a reachable combined state is μ_{M1} of a word in L(M1). []

K7 (R11 and C7 are T13). Admission reads Enforcer and writes none of its fields (R11) — D21 (ii); CalibrationAuthority drives Admission only through install/open/seal/close (C7) — D21 (iii); the write footprints are the rga_*, cal_* and identity vocabularies, disjoint by prefix. Hence every I-theorem holds of the identity component of the composed trace and every R-theorem of the scrutiny component, which is what Theorem 1 of the composed paper cites the two lemmas for. The proof of T13 is the proof the layer papers give twice by inspection; a fourth authority stacked by D21 inherits it without inspection.

What T13 does not say. It says nothing about information flow through the read channel: M2 may read everything in M1 and its *own* invariants may depend on M1's state in ways that a later change to M1 breaks. T13 is inheritance downward; there is no theorem upward, and there should not be — the upper layer is exactly the part that may need to change when the lower one does. It also does not cover the two side-by-side tables of the identity layer (the stage machine and the context envelope), which share writes to the stage's pc through GatePass; those compose by conjunction of guards, a synchronized product, and their invariants conjoin only because both tables' guards are checked before one effect — a fact the kernel's decide_pass enforces and T13 does not state. Nor does T13 hold *per step* of the kernel's calibration authority: _journal_atomic restores adm.lines and adm.sealed and truncates adm._events when a calibration attempt fails after Admission.seal has committed (rga/calibration.py:_journal_atomic, the admission_line branch), so the sentence in C7's proof that the only assignments in the file target the authority's own storage is false on that path, which no kernel test drives (CF3, reproduced: Admission's journal goes 12 → 13 → 12 across one attempt). Non-interference holds at *attempt* granularity — the lower machine is observed only between attempts — which is Lipton's reduction (1975), and D21 should say so.

6.2 Adequacy of a guard basis

D23 (Guard basis, violation space, kill relation). For a machine with target invariant Φ, a *guard basis* is a finite set G of named guard methods such that Φ holds of every reachable state with all of G intact. The *violation space* Viol(Φ) is the set of reachable-with-some-guard-deleted states violating Φ. The *mutant* M_{–g} replaces g by a no-op; the *kill relation* Kill ⊆ G × Tests holds when test t fails on M_{–g}.

T14 (Independence of guards and spanning). (a) *Independence.* A guard g is *load-bearing* for Φ iff Viol(Φ) ∩ Reach(M_{–g}) ≠ ∅; a test t with (g, t) ∈ Kill and t passing on M witnesses it. The delete-the-guard suite proves exactly this for each g singly (tests/test_rga_mutation.py; ../admissible/DRAFT.md §7, item 1). (b) *Spanning.* The suite certifies Φ — not merely each guard's

necessity — iff the union over g of the violation shapes reached by M_{–g} covers Viol(Φ) up to the equivalence "same guard would have refused it": i.e. iff every element of Viol(Φ) is refused by some guard whose deletion the suite exercises. Single-deletion mutants span Viol(Φ) iff no violation requires deleting two guards, i.e. iff Viol(Φ) has no element refused only by a conjunction.

Proof. (a) is the definition of a mutant plus the two-run discipline (intact: unreachable; deleted: reached). (b) If some $v \in$ Viol(Φ) is refused by two guards g, g' and by nothing else, then M_{–g} still refuses v (through g') and so does M_{–g'}; no single-deletion test reaches v, so no test witnesses that the pair is load-bearing for v, and the suite is silent on whether v is refused at all if both are later weakened. Conversely if every violation is refused by exactly one guard, deleting that guard reaches it and the suite's red run witnesses refusal. []

K8 (Where the kernel's suite stands under T14). The RGA and calibration suites prove (a) for every named guard. For (b), the kernel is explicit that some guards were *removed* because their targets were refused elsewhere ("clauses proved unreachable by construction were removed and derived instead", ../admissible/DRAFT.md §7, item 1; ../RGA/INVARIANTS.md §2, the paragraph following the transition table) — this is T14(b)'s condition being enforced by hand: a violation refused by two guards is a redundancy the discipline eliminates, so that every surviving guard is the unique refuser of its violation shapes. The separation-guard registry (tests/architecture/separation_guards.py, REQUIRED_SHAPES) is a hand-enumerated cover of a violation space; T14(b) says the completeness claim is a spanning claim, decidable when Viol(Φ) is derived from the transition table's complement (every edge the table lacks; every write the vocabulary lacks) rather than enumerated. §8 (N11) proposes that derivation.

Computed on the kernel (the compositional branch's kill matrix, reproduced; tests/test_custody.py, CF4): the 26-guard registry has every diagonal cell red and exactly one off-diagonal red — scenario s_guard_not_refused also reddens when _check_replay is deleted, so its predicate is not contained in its violation class — and two violations reachable only by deleting two guards at once that the JOINT table does not carry: a sealed line with k+1 samples (_guard_seal_complete $\wedge$ _guard_sample_count), and a sealed line, Admission.admissible, whose pinned refuter was refused *before* Open (_guard_pinned_before_open $\wedge$ _guard_not_refused), invisible to tainted because refused_at < sealed_at. These are T14(b)'s condition failing at exactly the points the theorem predicts.

What T14 does not say. It does not certify tests that pass for the wrong reason (a test that fails on M_{–g} because of a crash, not the violation): the kernel's four-valued verdict domain with ERROR as bottom (separation_guards.py) exists for exactly this, and T14 assumes it. It does not address guards whose violation is reachable only under an assumption failure (a lying harness), which are not guards but assumptions.

7. The custody game

The scrutiny layer's meaning is a move order: claims and refuters pinned before generation; the artifact hashed; a nonce drawn; a seed derived; trials run; a seal issued; escapes filed against the seal at fresh points of the same seed family. The layer papers call this separation of duty (R3) and seed-after-artifact (R10). It is a game whose move order is not enforced by a referee.

7.1 Players, moves, commitment

D24 (Custody game). A *custody game* has players Generator G, Harness H (runs refuters, reports verdicts), Finder F, Operator O (installs policy, adjudicates), and the Journal J (the machine). A *move* is an attempt in Att by a player. The *commitment order* $\lessdot_c$ on moves is generated by: $m \lessdot_c m'$ whenever m' carries a preimage-witnessed field whose preimage includes a root first created by m (D19). The

safety objective of the guards is the world property P_proc of §3.1: no state in which a necessity conjunct of admissible holds without its premises.

T15 (The guards are a winning strategy for safety in the one-shot game). For every finite sequence of moves by G, H, F, O in any interleaving, the state reached satisfies P_proc; equivalently, the guard set is a controller for the safety objective. Moreover the move order R3 requires — refuter pinned before any sample, sample i registered before stage i+1 attempted, seed after artifact — is enforced by three mechanisms, each of which T12 classifies: declared_at < opened_at by append-only membership (position, online-sound); V8 by recorded position (position, online-sound); R10 by preimage (commitment, online-unforgeable and rewrite-invariant).

Proof. Safety is T5 applied to the necessity conjuncts: every attempt that would establish one without its premises is refused or publishes; P_proc is the conjunction of the premises. The classification is K6. []

What T15 does not say. It says nothing about liveness: the guards can win by refusing everything, and the layer papers are explicit that nothing makes any gate reachable. It says nothing about the *repeated* game.

7.2 The repeated game and exposure

D25 (Exposure). The *exposure* of a line l is the set of journal facts the generator may have had in its context at the time of l's samples: by construction the excluded categories are not among them (_guard_sample_package, V7), but every earlier *published* outcome is — every rga_close with V1 naming which refuter refuted which claim on which sample of an earlier line of the same generator on the same body. Formally, expo(l) = { e ∈ r[0 : open(l)] : e is a published fault of a line l' with bindkey(l') = bindkey(l) }, and the *attempt index* of l is 1 + |{ l' : bindkey(l') = bindkey(l), open(l') < open(l) }|.

T16 (Exposure is a primary, not a rate, and it is monotone in the attempt index). expo(l) and the attempt index are pure functions of the record up to open(l); the attempt index is non-decreasing along a generator's retries on one bind key; and a seal issued on attempt n > 1 is a seal whose samples were produced by a generator that had, in the worst case, seen n − 1 published refutations of its own earlier outputs by the very refuters that then found nothing.

Proof. The definitions read the journal; the count is over earlier rga_open events with equal bind key (../RGA/INVARIANTS.md §1, Bind key); published closes carry their reason (rga/core.py:_close, reason=f"{refuter}@{version} refuted {claim} on sample {i}"). []

K9 (What the kernel says about this today). The layer paper: "Do not cite R3 as a theorem about Goodhart" (../RGA/INVARIANTS.md §7); the premise: the adaptive retry is the acknowledged unlabelled residue. T16 does not close it — a generator that has memorized a public refuter is B-side exposure no journal records — but it makes the *journaled* part of exposure a stamped primary (§8, N8), so that a seal on attempt 4 after three V1 closes naming the same refuter is visibly a different object from a seal on attempt 1, exactly as the seal already makes k = 1 visibly trivial (R4).

What T16 does not say. It does not say attempt 4 is worse; a generator may legitimately repair. It does not measure leakage — that would be a quantitative information-flow statement about the generator's context, which is A10/B1-side and unobservable. It carries a count and the reasons, and stops.

K11 (What the record does not order). Four facts about the scrutiny journal that T15 and T16 must not be read past, each reproduced on the kernel: no guard orders a trial on sample i against the registration of sample i+1 (trial checks index validity only; _guard_sample_order orders samples against later *stages*, never trials against later samples), and the kernel's own harness and bench

journal sample i's trial before sample i+1's bytes are hashed; the retry remainder of I7 lives on a stage of an item, and every attempt on a bind key is a new item, so I7 bounds admits per stage and gives no horizon on attempts; a tier-A escape may be filed at a *published* sample nonce — exactly the seed at which the seal's trial survived — and is accepted and established without being read as a divergence of the refuter (CF8); and an unestablished cal_run is journaled, so the *filed* set is finder-padded and any exposure component built on filings must read validity as of position.

8. What the theory licenses: twenty-eight kernel capabilities

Each capability is anchored to a code site, tiered by cost in the kernel's own culture — **formalize-only** (a paper change), **new-query** (a pure function over journal state the kernel already keeps; the cheapest and most in-culture tier, since every existing store query is one), **new-guard** (a refusal or publication added to an existing transition), **new-protocol** (a new root field or a new event) — and followed by the theorem it comes from and the sentence that says why it was not visible without the theorem. None of them changes what admissible returns today, except N28, which is a semantics change and says so; several change what a seal carries beside it. Each respects R11/C7: a new query writes nothing, and a new guard or protocol lives in the machine that owns the field.

N1. The deletion surface. *new-query.* deletion_surface(t) → set of (journal, position): the events whose removal would raise the standing of some line at position t — every established escape's cal_run+cal_replay pair, every rga_replay(diverged)+rga_refuse pair for a refuter some seal pinned, every cal_adjudicate(accept). Site: rga/calibration.py beside impeached, reading runs, adm.refused, adm.sealed. From T4.1. *Why unseen:* the residue was written as a description of a phenomenon; the theorem says the phenomenon is a set the kernel can enumerate, and an operator who sees "eleven events, on three lines, are the entire deletion surface" can anchor exactly those.

N2. The line-scoped standing certificate. *new-protocol* (a signed object, no new journal event). standing_certificate(id, t) = (H(roots of id's necessity events), |Δ_id(t)|, |r_t|) signed by the authority. Site: beside the v0.5 receipt path (tests/test_authenticated_receipt.py names it). From T11. *Why unseen:* the receipt was designed as a global anchor because "which events matter" had no answer; T11 gives the answer per line, and the result composes along depends_on in a way three global heads cannot (K5).

N3. Polarity typing of the event alphabet. *formalize-only* with a *new-query* polarity(query) → {σ |-> +,–,0,±}. Site: a table in metrics/SCHEMA.md and a pure function beside the store queries. From D6, T3. *Why unseen:* the residue paragraph of Theorem 2 lists directions by example; the polarity table derives them, and a future event type gets its polarity checked rather than argued.

N4. The joint reading, carried. *new-query* now; *new-protocol* if carried on the seal. power_joint(id) = max(0, 1 − Σ_j (1 − composite_j)) beside power_min, labelled bonferroni; and the same along depends_on: power_joint_closure(id). Site: rga/core.py:Admission.seal (composite list is in hand), check_dependencies. From T7, K3. *Why unseen:* the kernel refused products and chose min without asking what the Fréchet *lower* bound of a conjunction is; the answer is Boole's inequality, and its horizon (1/(1−p) edges) is a fact about the DAG that no per-edge floor can see.

N5. Kill-context primaries on the seal. *new-protocol* (fields in rga_seal). Per ledger claim: uncovered = |D \ ∪K_ρ|, and per refuter unique_kills = |K_ρ \ ∪_{ρ'≠ρ}K_{ρ'}|. Site: rga/core.py:Admission._claim_seal (the union is already computed; the differences are one line each). From T8. *Why unseen:* the seal carries kills and size because power is kills/size; the incidence structure was stored (killed_ids) for the union and never read for anything else. Two refuters with identical kill sets currently look like a union of two; with N5 the seal says one of them killed nothing the other did not.

N6. Hereditary standing. *new-query.* provenance(id) → {ancestor |-> (sealed, mediated, tainted, impeached, power_min)} over the transitive closure of depends_on, and hereditary_admissible(id) = admissible(id) ∧ ∀a ∈ ancestors: admissible(a), with the Bonferroni closure of N4. Site: rga/calibration.py:suspect (today one hop, one bit, "the consumer's fold"). From T7.1, T13 (a fold over the DAG is a stacked query). *Why unseen:* the one-hop design followed RFC 5280; the theorem says the fold is a pure query the authority can answer with the same non-interference as any other, and that its scalar shadow is the Bonferroni closure, not the minimum.

N7. Preimage witnesses for V8. *new-protocol.* Include a commitment H(rga_open event roots) in the package categories the generator receives, so that I11's package receipt binds "generated after Open" by preimage; and derive each sample's slot from the previous sample's nonce (ν_{i+1} includes H(ν_i | h_i)) so that interleaving (V8's second half) is preimage-witnessed. Sites: rga/core.py:_guard_open_before_generation, _guard_sample_order. Admission today reads a harness-reported package_categories set (B1), not the I11 receipt; N7 needs Admission to be given a read of the envelope's receipt — an upward read, allowed by D21 — against which the RGA-owned commitment is checked, and not something the identity layer reads from the scrutiny layer, which would reverse R11; if instead the harness supplies it, it is a B1 report and the witness is an assumption. *Tier, honestly:* new-protocol with one added trust assumption where the receipt path is not attested. From T12, K6. *Why unseen:* the two V8 guards were repaired in review to read the kernel's own position instead of a caller's, which is the online fix; T12 says the offline forgery of a position guard is a coherent rewrite, and that the same fact witnessed by a preimage is not.

N8. The exposure stamp. *new-query* now; *new-protocol* if carried. exposure(id) = (attempt_index, published_refutations: [(refuter, claim, sample, line)]) computed from earlier lines with the same bind key and their refuted trials (the companion reads the trials, not the close's reason string); stamped into cal_stamp beside track_records; with, if wanted, the count of published evaluation points of each pinned refuter on the bind key and a seed-variation flag, both read over *valid* runs only, since an unestablished filing is journaled (K11). Site: rga/calibration.py:_stamp. From T16. *Why unseen:* R3 was proved as a one-shot separation and the paper forbade citing it for the repeated game; T16 does not prove anything about the repeated game either — it makes the repeated game's journaled trace a primary on the seal, in the style of (agreeing, k).

N9. Trust bases as first-class, tiers derived. *formalize-only* now; *new-query* trust_base(seal) and trust_base(run). A seal's base is {B0, B1, B2, B7, B9, B10, B11} ∪ {B5 if any ledger refuter} ∪ {declared-(ε,N) if any bounded refuter} ∪ {B12}; a tier-A run's base is {B1, B2, B7, B10}; tier assignment is base(run) ⊆ base(seal). Site: rga/calibration.py:_file_run (tier = "A" if checker in pinned else "B" becomes the special case). From D7–D9 and the kernel's C1 (the definition of a tier-A escape). *Why unseen:* tier A was defined by pinned membership because that is the mechanism; the theorem says the *reason* it is automatic is base inclusion, and any future checker whose base is included (a second version of the same refuter under the same runtime hash, say) is tier A by the same theorem without a new rule.

N10. Pinned witness equivalence. *new-protocol.* Concordance today compares canonical witnesses by byte equality (B8) — the finest equivalence, hence the custodial choice: agreement under identity implies agreement under every coarser equivalence. A class may pin, at Open, a *witness equivalence* ≡_c given as a replayable, refusable program (a refuter of the claim "these two witnesses differ"), defaulting to identity. Site: rga/core.py:_witness_vector, _check_concordance, ClassAdmission. From §3's reading of B8 as [] over equivalences. *Why unseen:* "never an election" ruled out every *aggregation*; an equivalence is not an aggregation, it is a quotient, and the theory says the quotient must be a pinned instrument with the standing of a refuter — which is why it can be added without touching R4.

N11. The derived violation space. *tooling.* Generate Viol(Φ) from the transition table (every (pc, attempt) pair the table lacks; every write outside the vocabulary) and check that REQUIRED_SHAPES and the mutant registry span it. Site: tests/architecture/separation_guards.py, scripts/sabotage_admissible.py. From T14(b). *Why unseen:* the taxonomy was built by hand and its surjectivity checked; spanning is a different property, and it is decidable from the table.

N12. A two-dimensional demotion budget. *new-guard.* demoted(ρ, cls) compares charges against e_max; C5's track_record already carries seals_participated. Let the class declare e_max as a step function of seals_participated — still a declaration, still primaries, no rate. Site: rga/calibration.py:CalibrationClass, demoted. From D16's monus discipline and the gap inventory's "a refuter in 10,000 seals and one in 10 face the same budget". Beside it, charged_lines(ρ, cls) = (c′, n): the number of distinct lines carrying a tier-A escape *by* ρ *itself* and the seals ρ participated in — a pair of primaries with the per-checker certificate of T18(a), where the kernel's charges is a co-liability (K10). *Why unseen:* a computed ratio was forbidden, correctly; a declared curve over two primaries is a declaration on the class, like ε in bounded mode, not a computed rate.

N13. Semantic coverage as an adjudicated obligation. *formalize-only* now; *new-protocol* later. T9(c) shows id-containment is the weakest non-discretionary coverage that distinguishes escapes; a semantic coverage claim ("D covers the class of escape e") is a tier-B fact and belongs in the ratchet only through an adjudication event that names actor, reason and the (D, e) pair. Site: rga/calibration.py:_guard_install_covers, adjudicate. *Why unseen:* the gap inventory calls coverage "structurally undefined"; T9 says it is defined, that nothing non-discretionary is richer, and that the richer notion already has a home.

N14. A fourth authority, for free. *new-protocol.* Any machine stacked by D21 — a *provenance authority* answering N6 and issuing N2 certificates, journaled in its own prov_ *vocabulary, driving CalibrationAuthority only through its attempts — inherits I1–I17, R1–R13, C1–C7 by T13 with no new non-interference lemma. Site: a new module beside rga/calibration.py.* Why unseen:* each layer so far cost a lemma proved by inspection; T13 makes the lemma a construction.

The next fourteen come from the five branch formalizations this paper's review ran, each adversarially verified against the kernel; the ones marked *reproduced* were re-executed by a referee and again by the companion.

N15. A sorted floor, and a floor witness. *new-guard* / *new-query.* bound() accepts (ε = 1, N = 1), and composite = max(union, bounded), so a declared figure of 1.0 satisfies any p_min on a claim whose kernel-counted power is 0/|D| (CF6, reproduced in tests/test_custody.py). Let a class declare floor_sort ∈ {any, ledger} so the V5 gate compares the kernel-counted coordinate alone, and let floor_witness(id) name, per claim, which contributor realized the composite and on what base. Site: rga/core.py:_claim_seal, _check_power_floor. From D13, T6.1.

N16. exposed(id) and a chained attest. *new-query* / *new-protocol.* The exposed part of N1 as a per-line query (T4.1 as corrected), and an operator-invocable cal_attest(cls) event carrying, as of its position, the recomputed charge cells, impeached set, discredited set and refused set plus the previous attest's position, verified on replay as a stamp is. With chained attests every interior removal of a negative fact is refused by replay alone and the exposed set shrinks to the suffix after the last attest. Site: rga/calibration.py:_stamp, from_events. From T10(b), T11.

N17. The seam report, and two seams repaired. *new-query* / *new-guard.* replay_determinacy(): for every guard that reads a lower journal, whether replay evaluates it at the recorded cut, below it, or above it, and whether the verdict differs between the ends. One seam is a defect today: from_events re-checks _guard_audit_checker against escapes(cls) at the final registry, so an honest journal with an audit filed by a checker refused *later* is refused on rebuild (CF2, reproduced). Its mirror is CF14: the

cal_run branch of from_events omits the Admission.refused check _guard_run_checker makes live, so a filing by an already-refused checker is accepted on rebuild, and the same recorded cut lets rebuild re-check refused as the live machine did. The repair needs N19's cal_run.as_of and evaluates the guard at the filing's own recorded cut; evaluating below it (at the last earlier recorded cut) would accept an audit the live machine refused, trading a fail-closed seam for a fail-open one. Until N19 lands the seam stays, and it is fail-closed. Site: rga/calibration.py:from_events. From K6, §5.

N18. open_mediated(id). *new-query, contingent on N19.* CalibrationAuthority.open emits no event, so C6's open-time guard is never re-verified on rebuild, and under demotion_gate = carry a line opened around the authority with a demoted pinned refuter is mediated and admissible and replays clean (CF7, reproduced; under gate = seal CalSeal refuses it with E5). The guard is an as-of query — no pinned refuter demoted as of opened_at — but opened_at is a scrutiny position and demotion counts standing events, and today no cut orders the two (K11, CF11); without N19's cal_run.as_of the query is an over-approximation whose fail-closed form is "demoted as of the last standing event whose recorded cut is at most opened_at". Site: rga/calibration.py:mediated, _guard_open_demoted. From D8.

N19. Record the cut. *new-protocol.* Two root fields: cal_run.as_of (the scrutiny position at filing) and rga_seal.fcd_position. With them every cross-journal read the standing layer makes carries its live cut, and replay can re-drive the three machines as one merged fold at exactly the live cuts. A third field, rga_trial.fcd_position, was proposed by a branch and is not adopted: the record's only commitment for a sample is its rga_sample hash, against which a trial's order is already witnessed in the same journal; ordering a trial against the FCD call would order it against a harness report. Site: rga/calibration.py:_file_run, rga/core.py:seal. From K6, K11.

N20. A pinned trial regime. *new-guard.* trial_order ∈ {sequential, batch} on ClassAdmission, write-once at Open and carried on the seal; in batch mode _guard_trial_batch refuses any trial while |samples| < k, so no verdict of the line is journaled before the last sample's bytes are fixed — the ordering half of B4, position-witnessed. Site: rga/core.py:trial, ClassAdmission. From T15, K11.

N21. Two-phase audit points. *new-protocol.* audit_draw(line, claim, checker) journals a kernel-drawn nonce; audit_report(draw, verdict, witness) files against it; audit_exposure counts draws, with an unreported draw typed as the fail-closed direction. Today file_audit takes a finder-chosen nonce, so an auditor may evaluate privately at M points and file the m survivors (CF9). A one-transition version is impossible, since the verdict is an argument of the call that would draw the nonce. Site: rga/calibration.py:file_audit. From D28.

N22. An exposure budget per bind key. *new-guard.* n_max on CalibrationClass in the E9 pattern — explicit per class, no default — and _guard_open_exposure at CalOpen refusing a line whose attempt index on its bind key exceeds it, failing closed to *ask*. I7 gives no such horizon (K11). Site: rga/calibration.py:CalibrationClass, open. From T16.

N23. Journaled audit campaigns. *new-protocol.* cal_campaign pinning (line, claim, checker, N) with N nonces drawn from the authority's unpredictable source and journaled before any run; campaign_e_value(line, claim, checker, ρ_0) as a pure query valued 0 for any incomplete campaign — the standard e-process device; what is local is its mapping onto _file_run. Site: rga/calibration.py beside _file_run. From T18.

N24. Kill-set regression diff. *new-query.* regressions(ρ_old, ρ_new) = K(ρ_old, D) \ K(ρ_new, D') restricted to D ∩ D', computable because PowerRecord carries the kill set and not only its density; the bench's "successor power 0.9167 (was 0.9524)" decomposes into regressions, dropped ids (already journaled by _dropped_ids) and new-corpus misses. Site: rga/core.py:measure, rga/calibration.py:_dropped_ids. From D12, T8.

N25. Register the pair cuts; derive spanning. *tooling.* Add the two size-2 refuser sets of CF4 to JOINT with their single-deletion negatives; re-predicate s_guard_not_refused so its forbidden state lies in its violation class; and derive the Seal transition's violation classes from the abstract table (pc × |samples| vs k × guard bits) to assert every class with a minimal cut inside the basis is registered. In the separation harness, value expected-failure counts in the free commutative monoid on the case index rather than in N, so that the right total over the wrong case set fails. Site: tests/test_rga_mutation.py:JOINT, GUARDS; tests/architecture/separation_guards.py. From T14(b).

N26. Upward-stability typing of cross-layer reads. *tooling.* Every snapshot read an upper machine makes of a lower one is sound as a state invariant iff the fact is stable under the lower machine's own public attempts; positive membership in grow-only sets is, its negation is not and must be read as an as-of query. A test that types every such read (stage.pc == Passed, id ∈ store, refused, defect_ids) and fails when a lower-machine writer breaks a stability the upper machine assumed. Site: tests/test_rga_invariants.py beside R11RGAWritesNoFCDField. From T13's "what it does not say".

N27. Replayer provenance. *new-protocol.* A replayer field on rga_replay and cal_replay, journal-cited metadata that gates nothing, exactly as finder on cal_run. The trust base of a standing verdict then names the party whose divergent report it rests on, and a deployment that later distrusts one replayer can recompute which verdicts survive. Site: rga/core.py:replay, rga/calibration.py:replay_run. From D7, CF1.

N28. Un-revocation established, never asserted. *new-guard*, a semantics change. Two repairs, both of which would also rewrite §8.1's validity definition, C3 and C6 of the standing paper, and flip the test that pins today's behaviour (test_divergent_replay_discredits_monotonically). The minimal one is the ReplayEscape row's own semantics: _check_valid voids a run of a discredited or refused checker only when the run is *unestablished*, so that an escape established by identical replay is not un-established by a later single-party report on a different run; its cost is that an escape by a checker later caught diverging on replay keeps its standing. The principled one applies C1's rule to the falsifier: a discredit or refusal voids an *established* escape only when the divergence is itself established — a second, concordant divergent replay of the same run, journaled — so that un-revocation, like revocation, is entailed by a demonstration and not by a report. Either makes cal_discredit neutral on artifact standing and closes both channels of CF1; neither is applied here. Site: rga/calibration.py:_check_valid, replay_run; rga/core.py:replay. From T5 as corrected, D6.

The twenty-eight are not equal. N1, N3, N4, N6, N8, N16, N18 are pure queries a reviewer can add and test in the kernel's style — "every guard is a named method, every query a pure function" — without a protocol change, and each carries a theorem. N2, N7, N16 and N19 change what an adversary can forge offline, which is the residue the composed paper names as its largest. N5, N12, N15 and N24 change what a seal or a budget *says*, with primaries. N9, N10, N13, N20–N23 are the theory's account of design decisions, each with a path to a mechanism. N11, N25, N26 are methodology. N17 and N28 address two findings of §8a — N28 as a semantics change the standing paper would have to adopt in three places — and N27 names the party the first of them turns on.

8a. What the theory found in the kernel

Each row was predicted by a theorem of this paper or of one of its five branch formalizations and executed against the unmodified kernel by an adversarial referee (REVIEWS.md); every row except CF9, CF10 and CF13 is re-executed in the repository by tests/test_custody.py (CF14 by FindingCF14RefusedCheckerAcceptedOnRebuild) (the Finding *classes and PositionWitnessT12), and those three are read off the cited lines.* Direction* is the polarity of the finding: fail-open means an adversary can raise standing, fail-closed means an honest record is refused, spec means the code and a layer paper disagree.

CF1 — *Finding:* A divergent replay of *any* run by a checker discredits it and _check_valid voids every run of that checker, established or not. One party holding the sealed bytes files a junk tier-A escape and self-replays it divergently: the line is un-impeached, the pinned refuter un-demoted, nothing tainted, refused empty. The same shape through the other report: a divergent Admission.replay refuses a checker everywhere and voids its escapes at every position, so a tier-B checker's accepted escape against a line that never pinned it is voided by its refusal elsewhere, with no taint. The layer paper is divided: the ReplayEscape row of §8.2 says only *unestablished* escapes are void, while §8.1's validity definition, C3 and C6 void established ones too, and the code follows §8.1. · *Site:* rga/calibration.py:replay_run, _check_valid; rga/core.py:replay; ../RGA/INVARIANTS.md §8.1, §8.2 · *Direction:* fail-open (single-party report); layer-paper inconsistency · *Predicted by:* D6 (cal_discredit strictly positive, rga_refuse mixed), T5 as corrected · *Minimal repair:* N28

CF2 — *Finding:* from_events re-checks _guard_audit_checker against escapes(cls) at the final registry; an honest journal with an audit filed by a checker refused later is refused on rebuild. · *Site:* rga/calibration.py:from_events · *Direction:* fail-closed · *Predicted by:* K6 (as_of restricts one axis) · *Minimal repair:* N17

CF3 — *Finding:* A calibration attempt that fails after Admission.seal committed restores adm.lines and adm.sealed (and, on the install path, adm.policy) and truncates adm._events; C7's proof sentence — "The only assignments in the file target the private journal storage self._events, self.runs, self.discredited, self.exclusions and fields of Run" — is false on that path, and no kernel test drives it. · *Site:* rga/calibration.py:_journal_atomic (admission_line); ../RGA/PROOFS.md C7 · *Direction:* spec · *Predicted by:* T13's granularity remark · *Minimal repair:* state attempt granularity in C7; test the branch

CF4 — *Finding:* Two violations refused only by a pair of guards are absent from JOINT: a sealed line with k+1 samples; a sealed, Admission.admissible line whose pinned refuter was refused before Open (refused_at < sealed_at, so not tainted). One registered scenario reddens under an unrelated deletion. · *Site:* tests/test_rga_mutation.py:JOINT, s_guard_not_refused · *Direction:* adequacy · *Predicted by:* T14(b) · *Minimal repair:* N25

CF5 — *Finding:* cal_install carries coverage primaries that from_events never recomputes; deleting an escape before a later install replays clean, since the ratchet only gets easier. · *Site:* rga/calibration.py:from_events (cal_install) · *Direction:* fail-open (offline) · *Predicted by:* T4.1's anchor list: install is not an anchor · *Minimal repair:* recompute coverage on rebuild as stamps are

CF6 — *Finding:* bound($\varepsilon = 1$, $N = 1$) is accepted; max(union, bounded) then satisfies any p_min over a $0/|D|$ ledger. · *Site:* rga/core.py:bound, _claim_seal, _check_power_floor · *Direction:* fail-open (declared) · *Predicted by:* D13, T6.1 · *Minimal repair:* N15

CF7 — *Finding:* CalibrationAuthority.open emits no event; C6's open-time demotion gate is never re-verified. Under demotion_gate = carry, a line opened around the authority with a demoted pin is mediated, admissible, and replays clean (under gate = seal, CalSeal refuses it with E5). · *Site:* rga/calibration.py:open, mediated · *Direction:* fail-open (bypass; carry gate) · *Predicted by:* D8: the guard is a query, not an event · *Minimal repair:* N18, N19

CF8 — *Finding:* A tier-A escape may be filed at a published sample nonce — the seed of the seal's own surviving trial — and is established; the contradiction is not a replay divergence of the refuter, because R5 fires only through Admission.replay. · *Site:* rga/calibration.py:_guard_run_seed · *Direction:* unread evidence · *Predicted by:* K11 · *Minimal repair:* refuse a filed nonce equal to a sample nonce, or read the contradiction as a divergence

CF9 — *Finding:* Audit nonces are finder-chosen, so audit_exposure counts chosen points; an auditor may evaluate privately at M and file m survivors. · *Site:* rga/calibration.py:file_audit · *Direction:* selection

· *Predicted by:* T18(a) · *Minimal repair:* N21

CF10 — *Finding:* A charge cell is a co-liability of every refuter pinned on the claim, not a per-refuter miss certificate; the bench's spec-weak carries spec-strong's charges. · *Site:* rga/calibration.py:charge_cells; eval/bench/RESULTS.md · *Direction:* semantics · *Predicted by:* K10 · *Minimal repair:* document the reading; N12's per-checker certificate

CF11 — *Finding:* The rga_seal event carries no position; only cal_stamp.sealed_at anchors the scrutiny journal's length, so an unmediated seal is unanchored and a refusal group before it deletes clean. as_of restricts only the scrutiny axis. The comment at rga/calibration.py:from_events (cal_stamp branch) that deleting a stamp "can only lower standing to IR" is stale for interior stamps: a stamp for *any* unmediated seal can be inserted at its seal-ordered position with later as_of fields renumbered, and deleting a stamp together with the escape it anchors raises the earlier line. · *Site:* rga/core.py:seal; rga/calibration.py:from_events · *Direction:* fail-open (offline) · *Predicted by:* K6 · *Minimal repair:* N19

CF12 — *Finding:* Recorded positions are roots on replay, range-checked only; rewriting rga_open.fcd_position downward satisfies the before-generation guard whatever the true order. · *Site:* rga/core.py:Admission.from_events, _guard_open_before_generation · *Direction:* fail-open (offline) · *Predicted by:* T12(c) · *Minimal repair:* N7

CF13 — *Finding:* Nothing orders a trial on sample i against the *registration* of sample i+1: trial checks index validity only, and the kernel's harness and bench journal sample i's trial before sample i+1's bytes are hashed. (Against the FCD stage the order is unwitnessed too, but the stage is a report; the sample hash is the commitment.) Unestablished cal_runs are journaled. · *Site:* rga/core.py:trial, _guard_sample_order · *Direction:* unwitnessed · *Predicted by:* K11 · *Minimal repair:* N20

CF14 — *Finding:* from_events (cal_run branch) checks that the checker is declared and not discredited but not that it is in Admission.refused; a filing by an already-refused checker is refused live (_guard_run_checker) and accepted on rebuild. Standing-neutral (_check_valid voids it), but a fail-open replay seam — the mirror of CF2 — and a record no world in ∩B contains, on which every []_B is vacuous (D8). · *Site:* rga/calibration.py:from_events, _guard_run_checker · *Direction:* fail-open (replay seam) · *Predicted by:* D8's vacuity premise · *Minimal repair:* re-check refused on rebuild, as-of the filing's cut (N17, N19)

None of these contradicts a theorem of the layer papers as those theorems are stated: CF1 is C6 ("falling when a checker's discredit or refusal degrades validity") doing what C6 says, against one row of its own transition table; CF2–CF14 are residues the papers either name or would name. What the table adds is the *sign* of each — the theory's contribution is that every row has one — and the observation that C3's non-discretion is one-sided: revocation is entailed by a replayable proof, un-revocation by a single-party report through either of two channels (a divergent cal_replay, with no collateral lowering, or a divergent Admission.replay, which also taints every seal that pinned the checker after sealing). Those two channels are the only online paths by which a report rather than a demonstration raises standing (T5).

9. Novelty ledger

The layer papers keep a ledger of what is old. This one was refereed nine times — fifteen adversarial reviews of its five branch formalizations, three of the unified draft, a focused re-check, the pull request's automated review, two seven-referee re-reviews of the pull request head, and three internal six-reviewer adversarial re-reviews (REVIEWS.md) — and the literature referees returned *derivative* on every branch and *major revision* on the whole. Both verdicts are accepted here and the ledger is written to them, in three columns.

Known mathematics, applied. The custodial reading []_B is the knowledge operator of the runs-and-systems framework with the record as the verifier's local state (Halpern & Moses 1990; Fagin, Halpern, Moses & Vardi 1995), relativized to a trust base; its ancestor in databases is the *certain answers* semantics over incomplete information (Imielinski & Lipski 1984). T2 is stability of knowledge of stable facts under perfect recall (Chandy & Misra 1986); that positive facts persist along a growing record and negated ones do not is Kripke persistence (1965), and its use for a machine that reads a grow-only set is the CALM principle (Hellerstein 2010; Ameloot, Neven & Van den Bussche 2011) with Dedalus's temporal stratification of negation (Alvaro et al. 2011) as the as_of discipline and Blazes' sealing (Alvaro, Conway, Hellerstein & Maier 2014) as the anchoring stamp. Demonstrations are proof objects; a standing query's negative conjuncts are Doyle's out-list (1979), their validity degradation is Dung's grounded semantics (1995), and their reinstatement by attacking the attacker — CF1's mechanism — is Pollock's undercutting defeat (1987), and its assurance-case form, a confidence claim undercut by defeaters, is eliminative argumentation (Bloomfield & Rushby 2024). The roots/witnesses/length triple an anchor pins (T4.2, T11) is the correctness/completeness/freshness triple of authenticated query processing over adversarially held data (Devanbu, Gertz, Martel & Stubblebine 2003; Li, Hadjieleftheriou, Kollios & Reyzin 2006), with non-membership proofs as a distinct object from membership proofs (Kocher 1998; Naor & Nissim 1998); and the standing-query shape valid = issued ∧ ¬revoked, where deleting a revocation raises standing and a CRL number is the length witness, is X.509 (RFC 5280), which the layer papers already cite as impeachment's ancestor, with the per-subject signed status of N2 an OCSP response (RFC 2560) and its demonstration count a degenerate accumulator (Camenisch & Lysyanskaya 2002). Replay over a tamper-evident log with a detectably-faulty/detectably-ignorant split is PeerReview (Haeberlen, Kouznetsov & Druschel 2007). The support (D26) is de Kleer's ATMS label (1986) on the positive side and where-provenance (Buneman, Khanna & Tan 2001) on its determination clause; anchoring by later recomputation is the forward closure of dependency provenance (Cheney, Ahmed & Acar 2011; lineage, Cui, Widom & Wiener 2000), and a recomputed count that detects an earlier deletion is a control total. Guarded partial folds and prefix-closed languages are Alpern–Schneider safety (1985). The bounds of T6 and T7 are the Boole–Fréchet inequalities (Fréchet 1935; sharpness by Hailperin 1965), attained at the Fréchet–Hoeffding upper copula (Hoeffding 1940; Fréchet 1951); Cornell (1967) and Ditlevsen (1979) are their structural-reliability form for series and parallel systems, and the ideal sum of T7.2 is the cut-set bound of a coherent system (Esary & Proschan 1963; Barlow & Proschan 1975; NUREG-0492). Formal contexts (D15, T8) are Ganter–Wille. The gap-free sequence number of T11's length witness is Schneier & Kelsey (1999) and Crosby & Wallach (2009), whose consistency proofs are how two heads of one log compose (RFC 6962). T12's preimage witness is hash-pointer causality (Haber & Stornetta 1991), the commit-then-challenge argument of every Σ-protocol (Blum 1983; Fiat & Shamir 1986), and its position/preimage dichotomy is the timestamp/nonce dichotomy for freshness (Denning & Sacco 1981; Abadi & Needham 1996). T13 is the frame rule (O'Hearn, Reynolds & Yang 2001) in its abstract-separation-algebra form (Calcagno, O'Hearn & Yang 2007), with attempt-granular atomicity by Lipton's reduction (1975). T14(a) is independence of postulates by deletion (Hilbert 1899; Huntington 1904), and on the pair cuts Jia & Harman's higher-order mutants (2009) with Budd & Angluin's adequacy caveat (1982). T18 is the architecture of risk-limiting audits (Stark 2008; Waudby-Smith, Stark & Ramdas 2021) with the zero-failure bound of Hamlet (1987) and Miller et al. (1992) and the e-process devices of Ville (1939), Shafer (2021), Vovk & Wang (2021) and Grünwald, de Heide & Koolen (2024). T16's exposure lives in Denning's lattice (1976) under Lamport's happens-before (1978).

Local, and load-bearing. What is not in those sources is the application of all of them to one object, and the findings that produced:

1. That for a *replayed guarded fold* the completeness obligation of authenticated query processing falls only on the negated demonstrations: replay re-derives every positive atom from roots for free (T1, T2), so the anchor's job is the negative half and the length — the sign locates the obligation, and the first draft's claim that the sign itself was new was wrong.

2. That standing consumes only the *presence* of witnesses, so a bare count of valid demonstrations, not an accumulator with non-membership witnesses, suffices as the negative component of a per-line certificate (T11); the certificate is OCSP-shaped, and its composition along dependencies is length-equality *given* a journal identity that must still be supplied (K5).

3. The nonce/timestamp dichotomy read onto every temporal guard in the kernel, with the reproduced fact that a recorded position is a root replay only range-checks (T12, CF12).

4. CF1: the standing paper's C3 — "validity degrades by journal facts, never by discretion" — holds for revocation and fails for reinstatement, which a single-party report entails through either of two channels (D6, T5, N28); the signs of cal_discredit (strictly positive, by a second-order route) and rga_refuse (mixed), and the layer paper's own inconsistency between one transition-table row and its validity definition, were found by computing the polarity table rather than reading the prose.

5. Anchoring by recomputation as a property of *which* kernel events — same-class stamps, E5 closes, audits by the escape's checker, installs a refusal let pass — enumerated from from_events, with the two pair cuts and the rollback path the mutation and non-interference arguments did not see (T4.1, T10b, CF3, CF4), and the corrected deletable surface after three rounds of being wrong about it.

6. The reading of min over conjuncts as the upper Boole–Fréchet bound wearing a lower bound's name, the Bonferroni horizon along a dependency DAG, and the sortless floor (K3, T7.1, CF6): the cut-set bound of system reliability, not previously applied to mutation-measured kill densities, which no mutation-testing source composes across artifacts.

On the word *custody*. In evidence law and digital forensics a *chain of custody* certifies integrity by documented handling among trusted custodians (Prayudi & Sn 2015) — the opposite trust assumption from D7, where the custodians are the adversary and certification is by replay. The theory is chain of custody with the trust assumption inverted, and the name is kept for that reason; no prior use of *custodial reading* or *custodial semantics* was found, and the financial sense of self-custody is unrelated.

Conjectural. (i) That the polarity table of N3 is complete for admissible — the first draft mislabelled rga_refuse and the second cal_discredit, so the table has been wrong twice. (ii) That N7's nonce chaining does not weaken B9. (iii) That T14(b)'s spanning condition is decidable for the envelope machine. (iv) That the anchoring relation of T4.1 is complete. It was enumerated from the readers in from_events — stamps, E5 closes, audit filings, installs — after three enumerations were refuted; a derivation from the root-field dependency graph, into which derived_defect_id's position argument would enter, is N11's job and has not been done.

10. Limits, and what is not claimed

Everything the layer papers do not claim, this paper does not claim, and it adds nothing to the trusted base. Specifically:

Truth. []_B P_proc is a necessity about *procedure*; no world property in this paper is "the artifact is correct", and the custodial reading of correctness is false at every record (a world in which the artifact is wrong and every report was honest is consistent with every record the kernel can produce). Survival is not correctness; impeachment's absence is not soundness.

Readings. Every probability in §4 is under a within-model reading the reader chose (D14). T6 and T7 say which aggregators are sound under every coupling *of those readings*; they say nothing about the

coupling of D to a generator's real defects, which is B6 and remains an empirical hypothesis the composed paper's real-defect study attacks and does not settle.

Assumptions. [[·]]_B is defined relative to a trust base; every theorem here holds under B = {A0–A13, B0–B14} and the calibration layer's, and a failure of any assumption is a world outside [[·]]_B about which the theorems say nothing. T4(b) locates the A10/B1 boundary as the boundary of [[·]]_B; it does not move it.

Anchoring. T11's certificate must be signed and published; signer custody, registry monotonicity, and the first anchor's own past are B14 and the composed paper's residue, unchanged. T12 makes offline forgery of a position guard cost a coherent rewrite of an anchored root; it does not make it impossible without the anchor.

Liveness. Nothing here makes any gate reachable; T15 is safety only, and a strategy that refuses everything satisfies it.

Completeness of polarity and of the fault table. Conjectural, §9. Capabilities are labelled N1–N28 and findings CF1–CF14; the CF prefix keeps them apart from the kernel's standing theorems C1–C7 and the identity layer's faults F1–F10.

The word. §1.1 recommends *non-replayable* for *stochastic*; nothing proved in any paper depends on the substitution.

This document. It was drafted from the kernel and the layer papers by an assistant at the author's request; it was refereed by nine rounds (REVIEWS.md) that found a fatal inconsistency in its first semantics, refuted two of its enumerations, and corrected two theorem statements and the companion's anchor and closure enumeration and its power-horizon query, all applied here, and adopted by the author for this release. Its theorems have proofs; its capabilities have sites; the findings and the computed capabilities have tests, the theorems do not. In the kernel's own discipline that leaves every theorem here a candidate until it carries a test, and the executable companion (custody.py, tests/test_custody.py) is the beginning of that obligation, not its discharge.

References

Abadi, M., and Needham, R. Prudent engineering practice for cryptographic protocols. IEEE Transactions on Software Engineering, 1996.

Alpern, B., and Schneider, F. B. Defining liveness. Information Processing Letters, 1985.

Alvaro, P., et al. Dedalus: Datalog in time and space. Datalog Reloaded, 2011.

Alvaro, P., Conway, N., Hellerstein, J. M., and Maier, D. Blazes: Coordination analysis for distributed programs. IEEE International Conference on Data Engineering (ICDE), 2014.

Ameloot, T. J., Neven, F., and Van den Bussche, J. Relational transducers for declarative networking. ACM Symposium on Principles of Database Systems (PODS), 2011.

Barlow, R. E., and Proschan, F. Statistical Theory of Reliability and Life Testing. Holt, Rinehart and Winston, 1975.

Bloomfield, R., and Rushby, J. Defeaters and eliminative argumentation in Assurance 2.0. arXiv:2405.15800, 2024.

Blum, M. Coin flipping by telephone: a protocol for solving impossible problems. ACM SIGACT News, 1983.

Budd, T. A., and Angluin, D. Two notions of correctness and their relation to testing. Acta Informatica, 1982.

Buneman, P., Khanna, S., and Tan, W.-C. Why and where: A characterization of data provenance. International Conference on Database Theory (ICDT), 2001.

Calcagno, C., O'Hearn, P. W., and Yang, H. Local action and abstract separation logic. IEEE Symposium on Logic in Computer Science (LICS), 2007.

Camenisch, J., and Lysyanskaya, A. Dynamic accumulators and application to efficient revocation of anonymous credentials. CRYPTO, 2002.

Chandy, K. M., and Misra, J. How processes learn. Distributed Computing, 1986.

Cheney, J., Ahmed, A., and Acar, U. A. Provenance as dependency analysis. Mathematical Structures in Computer Science, 2011.

Cooper, D., et al. RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. IETF, 2008.

Cornell, C. A. Bounds on the reliability of structural systems. Journal of the Structural Division (ASCE), 1967.

Crosby, S. A., and Wallach, D. S. Efficient data structures for tamper-evident logging. USENIX Security Symposium, 2009.

Cui, Y., Widom, J., and Wiener, J. L. Tracing the lineage of view data in a warehousing environment. ACM Transactions on Database Systems, 2000.

de Kleer, J. An assumption-based TMS. Artificial Intelligence, 1986.

Denning, D. E. A lattice model of secure information flow. Communications of the ACM, 1976.

Denning, D. E., and Sacco, G. M. Timestamps in key distribution protocols. Communications of the ACM, 1981.

Devanbu, P., Gertz, M., Martel, C., and Stubblebine, S. G. Authentic data publication over the internet. Journal of Computer Security, 2003.

Ditlevsen, O. Narrow reliability bounds for structural systems. Journal of Structural Mechanics, 1979.

Doyle, J. A truth maintenance system. Artificial Intelligence, 1979.

Dung, P. M. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. Artificial Intelligence, 1995.

Esary, J. D., and Proschan, F. Coherent structures of non-identical components. Technometrics, 1963.

Fagin, R., Halpern, J. Y., Moses, Y., and Vardi, M. Y. Reasoning About Knowledge. MIT Press, 1995.

Fiat, A., and Shamir, A. How to prove yourself: practical solutions to identification and signature problems. CRYPTO, 1986.

Fréchet, M. Généralisation du théorème des probabilités totales. Fundamenta Mathematicae, 1935.

Fréchet, M. Sur les tableaux de corrélation dont les marges sont données. Annales de l'Université de Lyon, 1951.

Ganter, B., and Wille, R. Formal Concept Analysis: Mathematical Foundations. Springer, 1999.

Grünwald, P., de Heide, R., and Koolen, W. Safe testing. Journal of the Royal Statistical Society: Series B, 2024.

Haber, S., and Stornetta, W. S. How to time-stamp a digital document. Journal of Cryptology, 1991.

Haeberlen, A., Kouznetsov, P., and Druschel, P. PeerReview: Practical accountability for distributed systems. ACM Symposium on Operating Systems Principles (SOSP), 2007.

Hailperin, T. Best possible inequalities for the probability of a logical function of events. American Mathematical Monthly, 1965.

Halpern, J. Y., and Moses, Y. Knowledge and common knowledge in a distributed environment. Journal of the ACM, 1990.

Hamlet, R. G. Probable correctness theory. Information Processing Letters, 1987.

Hellerstein, J. M. The declarative imperative: experiences and conjectures in distributed logic. ACM SIGMOD Record, 2010.

Hilbert, D. Grundlagen der Geometrie. Teubner, 1899.

Hoeffding, W. Masstabinvariante Korrelationstheorie. Schriften des Mathematischen Instituts der Universität Berlin, 1940.

Huntington, E. V. Sets of independent postulates for the algebra of logic. Transactions of the American Mathematical Society, 1904.

Imielinski, T., and Lipski, W. Incomplete information in relational databases. Journal of the ACM, 1984.

Jia, Y., and Harman, M. Higher order mutation testing. Information and Software Technology, 2009.

Kocher, P. C. On certificate revocation and validation. Financial Cryptography (FC), 1998.

Kripke, S. A. Semantical analysis of intuitionistic logic I. Formal Systems and Recursive Functions, 1965.

Lamport, L. Time, clocks, and the ordering of events in a distributed system. Communications of the ACM, 1978.

Laurie, B., Langley, A., and Kasper, E. RFC 6962: Certificate Transparency. IETF, 2013.

Li, F., Hadjieleftheriou, M., Kollios, G., and Reyzin, L. Dynamic authenticated index structures for outsourced databases. ACM SIGMOD International Conference on Management of Data, 2006.

Lipton, R. J. Reduction: a method of proving properties of parallel programs. Communications of the ACM, 1975.

Miller, K. W., et al. Estimating the probability of failure when testing reveals no failures. IEEE Transactions on Software Engineering, 1992.

Myers, M., et al. RFC 2560: X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP. IETF, 1999.

Naor, M., and Nissim, K. Certificate revocation and certificate update. USENIX Security Symposium, 1998.

O'Hearn, P. W., Reynolds, J. C., and Yang, H. Local reasoning about programs that alter data structures. Computer Science Logic (CSL), 2001.

Pollock, J. L. Defeasible reasoning. Cognitive Science, 1987.

Prayudi, Y., and Sn, A. Digital chain of custody: State of the art. International Journal of Computer Applications, 2015.

Schneier, B., and Kelsey, J. Secure audit logs to support computer forensics. ACM Transactions on Information and System Security, 1999.

Shafer, G. Testing by betting: A strategy for statistical and scientific communication. Journal of the Royal Statistical Society: Series A, 2021.

Stark, P. B. Conservative statistical post-election audits. Annals of Applied Statistics, 2008.

Vesely, W. E., et al. Fault Tree Handbook (NUREG-0492). U.S. Nuclear Regulatory Commission, 1981.

Ville, J. Étude critique de la notion de collectif. Gauthier-Villars, 1939.

Vovk, V., and Wang, R. E-values: Calibration, combination and applications. Annals of Statistics, 2021.

Waudby-Smith, I., Stark, P. B., and Ramdas, A. RiLACS: Risk-limiting audits via confidence sequences. E-Vote-ID, 2021.